

:::success

二次开发模版地址: <https://gitee.com/shuiruohanyu/yikang-elder-care.git>

完整代码仓库地址: <https://gitee.com/shuiruohanyu/ykyl>

接口文档地址: <https://apifox.com/apidoc/shared-4a97787e-b0e6-46ec-ae1a-5fc0da5e192e>

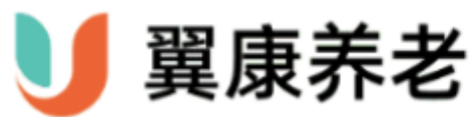
UI设计图地址: <https://codesign.qq.com/s/YDgGjYwLpa0wEVQ>

:::

项目介绍

** 随着全球人口老龄化的加速，养老服务行业面临着越来越多的挑战和机遇。为了满足日益增长的需求，养老机构需要不断提升服务质量，提高工作效率，同时确保老人的生活质量。在这个背景下，员工端应运而生，作为养老系统的重要组成部分，为员工提供了一个全新的工作平台。**





护理工作台



护理任务



护理记录

设备管理



设备安装



设备控制



我的设备



报警设备

用爱服务每一位老人

登录成功



首页



负责老人



护理工作台



我的

- 知识体系-通过项目学到什么?

1. 鸿蒙三层架构 hsp 模式
2. 鸿蒙三方常用插件使用
3. 华为手机号授权登录
4. 华为云IoTDA 接入使用
5. 低功耗蓝牙设备通信流程
6. 物联网设备的接入开发

- 项目总结亮点

1. 物联网养老项目-员工使用
2. 提供给养老院护理员使用

3. 核心功能-华为认证登录-护理员1对多管理老人的护理和照看，通过万物互联等物联网设备实现如给老人测量血糖，上传血糖数据，门禁开门/关门，蓝牙/Wifi等HarmonyOS能力集的应用
4. 三层架构-har + 多hsp共享包 + 入口hap包的综合应用

项目创建

并不是所有项目都需要用三层

- 工具类小项目 单层
 - 中小型项目-har + hap
 - 中大型-basic(基础能力提供层)-features(特性层)-product(入口层)
- 三层架构设计概述

文档中心

HarmonyOS应用的分层架构主要包括三个层次：产品定制层、基础特性层和公共能力层，为开发者构建了一个清晰、高效、可扩展的设计架构。

公共能力-hsp-har(静态共享包-会多份拷贝)

基础特性-hsp-动态共享包- 不论如何打包，永远只有一份
产品定制-hap入口-具备ability-具备窗口能力

• HAR静态共享包

HAR (Harmony Archive) 是静态共享包，可以包含代码、C++库、资源和配置文件。通过HAR可以实现多个模块或多个工程共享ArkUI组件、资源等相关代码。

使用场景：

- 作为三方库，发布到[OHPM](#)私仓，供公司内部其他应用使用。
- 作为三方库，发布到[OHPM](#)中心仓，供其他应用使用。

限制：

- HAR不支持在设备上单独安装/运行，只能作为应用模块的依赖项被引用。
- HAR不支持在配置文件中声明[UIAbility](#)组件与[ExtensionAbility](#)组件。
- HAR不支持在配置文件中声明[pages](#)页面，但是可以包含pages页面，并通过[命名路由](#)的方式进行跳转。
- HAR不支持引用AppScope目录中的资源。在编译构建时，AppScope中的内容不会打包到HAR中，因此会导致HAR资源引用失败。
- HAR可以依赖其他HAR，但不支持循环依赖，也不支持依赖传递。

• HSP动态共享包

文档中心

HSP (Harmony Shared Package) 是动态共享包，可以包含代码、C++库、资源和配置文件，通过HSP可以实现应用内的代码和资源的共享。HSP不支持独立发布，而是跟随其宿主应用的APP包一起发布，与宿主应用同进程，具有相同的包名和生命周期。

使用场景：

- 多个HAP/HSP共用的代码和资源放在同一个HSP中，可以提高代码、资源的可重用性和可维护性，同时编译打包时也只保留一份HSP代码和资源，能够有效控制应用包大小。
- HSP在运行时按需加载，有助于提升应用性能。

限制：

- HSP不支持在设备上单独安装/运行，需要与依赖该HSP的HAP一起安装/运行。HSP的版本号必须与HAP版本号一致。
- HSP不支持在配置文件中声明[UIAbility](#)组件与[ExtensionAbility](#)组件。
- HSP可以依赖其他HAR或HSP，但不支持循环依赖，也不支持依赖传递。

- 总结

:::info

hsp 和 har 区别?

相同点:

1. 模块间资源共享
2. 不支持在设备上单独安装/运行
3. 不支持在配置文件中声明 [UIAbility](#) 组件与 [文档中心](#) 组件
4. 不支持循环依赖, 也不支持依赖传递

不同点:

1. har 发布二方或三方仓使用 (hsp 不可以)
 2. har 中不支持在配置文件中声明 [pages](#) 页面
 3. hsp 中支持在配置文件中声明 [pages](#) 页面
 4. hsp 多模块依赖只会打包一份
 5. hsp 支持按需加载
- ...

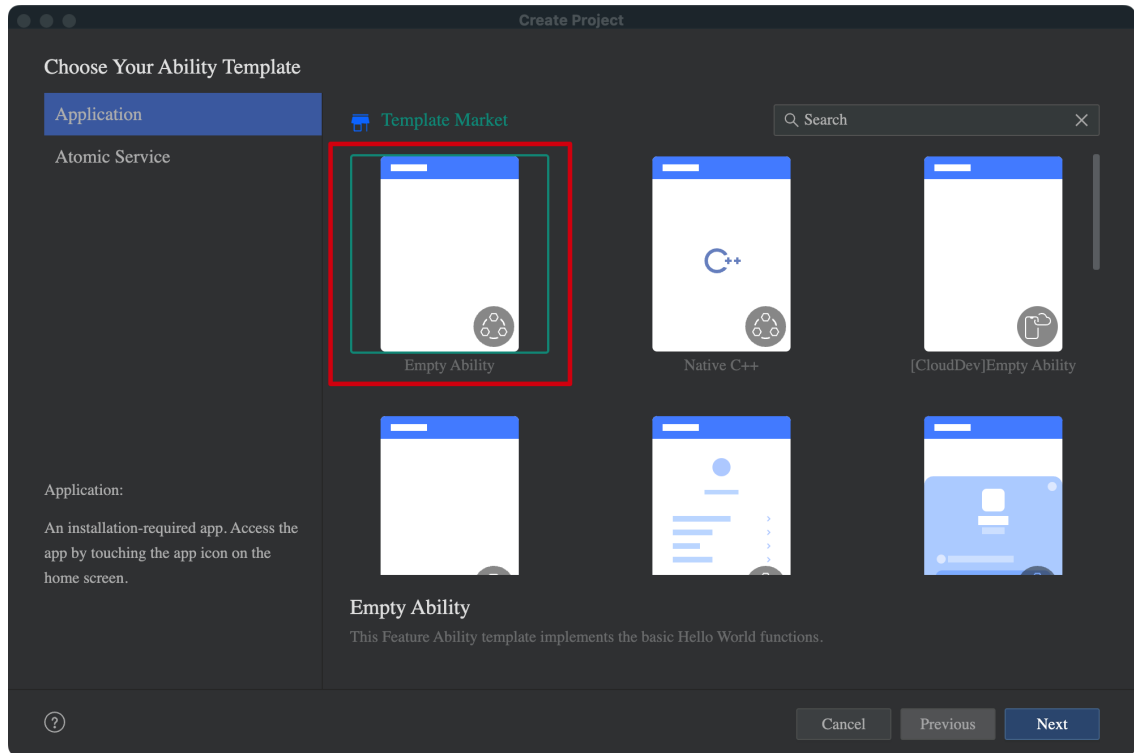
- 三层架构体验

步骤:

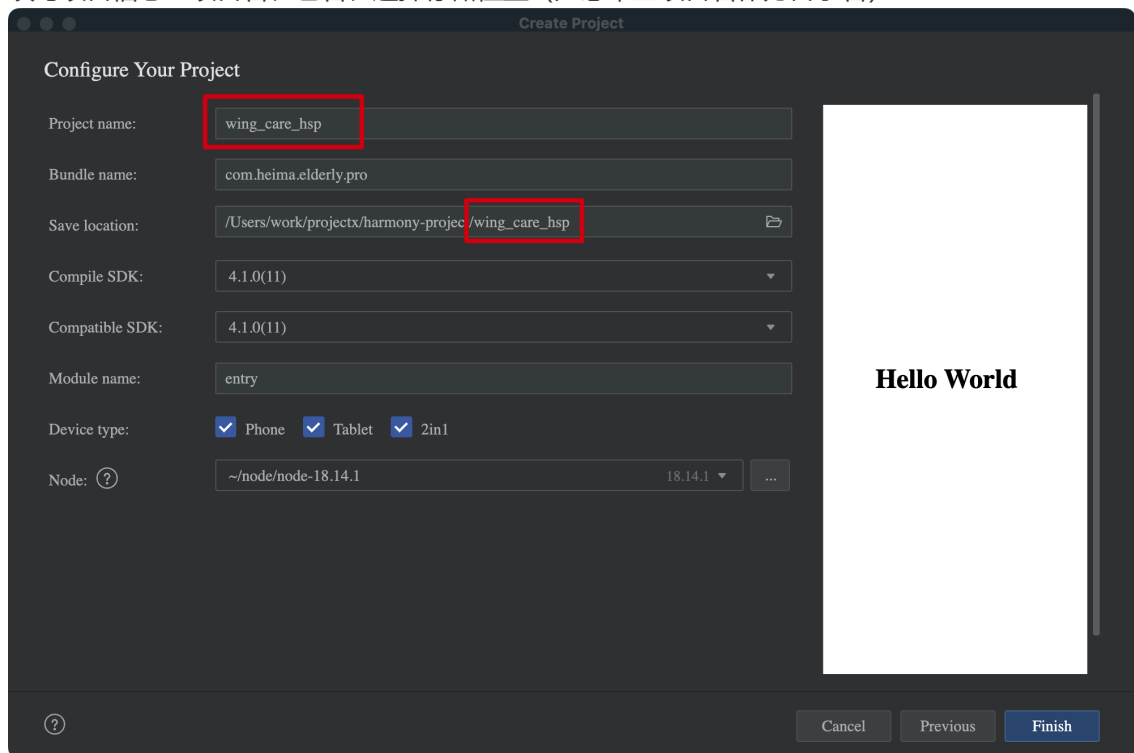
1. 选择文件-新建项目



2. 选择Empty Ability, 然后 Next



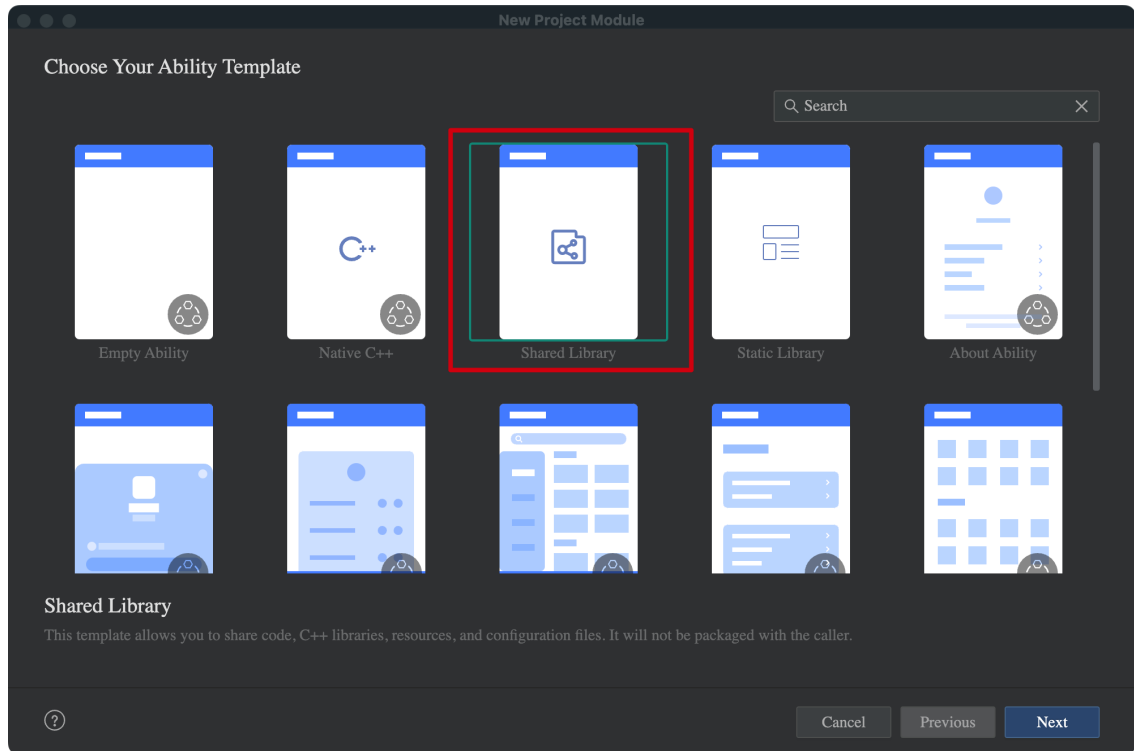
3. 填写项目信息: 项目名、包名、选择存储位置 (注意带上项目名作为目录名)



演示包名: com.heima.elderly.pro

4. 新建common、features、product 目录

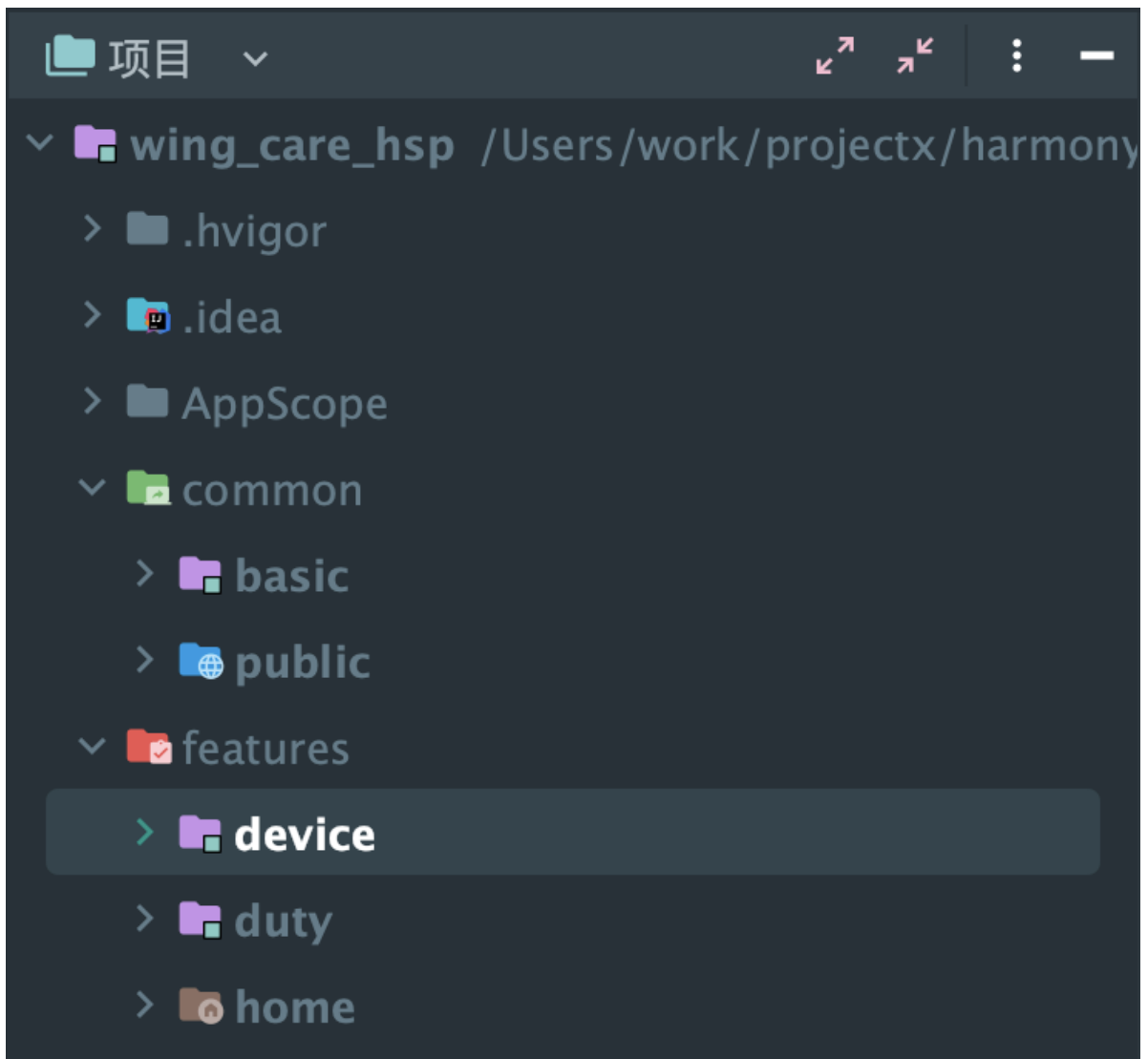
5. 在 common 下新建一个叫 basic 公共模块 Shared Library 包



6. 在 features 下新建home、duty、stage、mine、message、device业务模块 Shared Library 包

7. 把默认 entry 包改名为 phone 放到product 目录下

完成后目录结构，如图：



- >  message
- >  mine
- >  stage
- >  hvisor
- >  oh_modules
- ▼  product
 - >  phone
-  .gitignore
- {5,} build-profile.json5
-  hvisorfile.ts
-  hvisorw
-  hvisorw.bat
- </> local.properties
- {5,} oh-package.json5
- {5,} oh-package-lock.json5

8. 运行项目（修改运行调试配置）

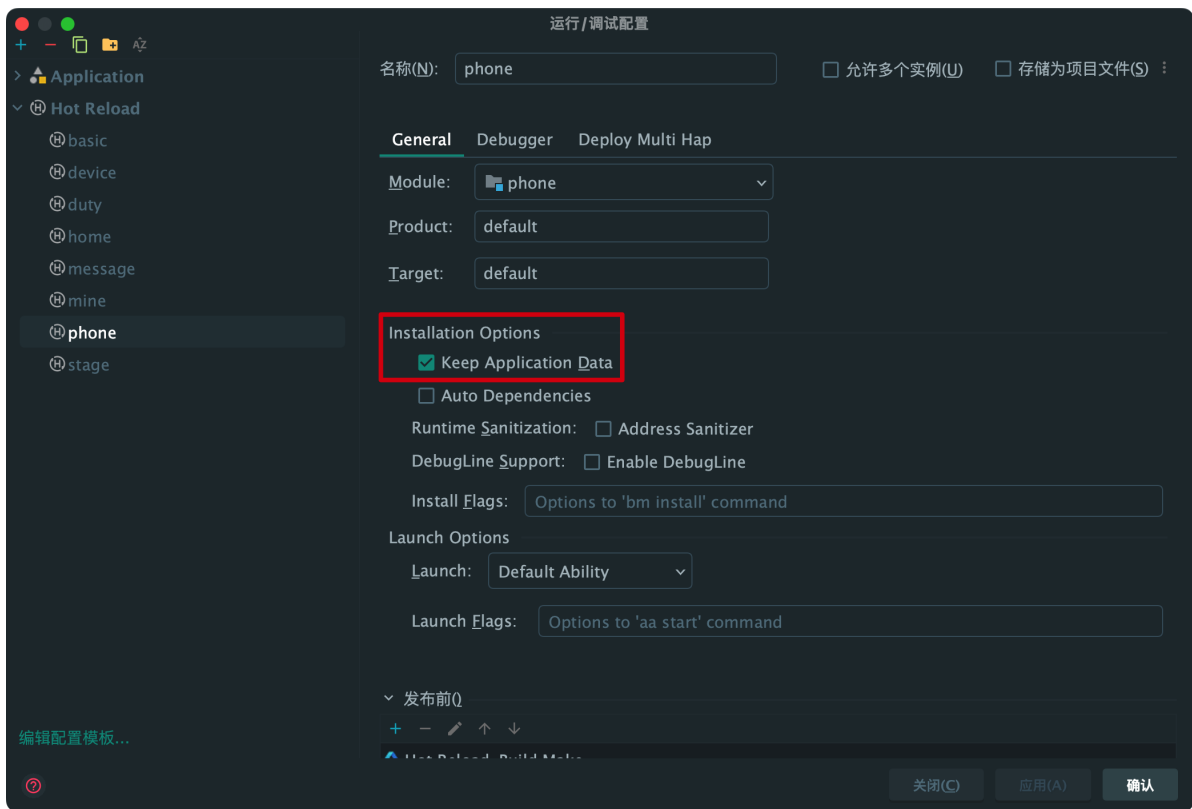
项目运行

配置运行项目：

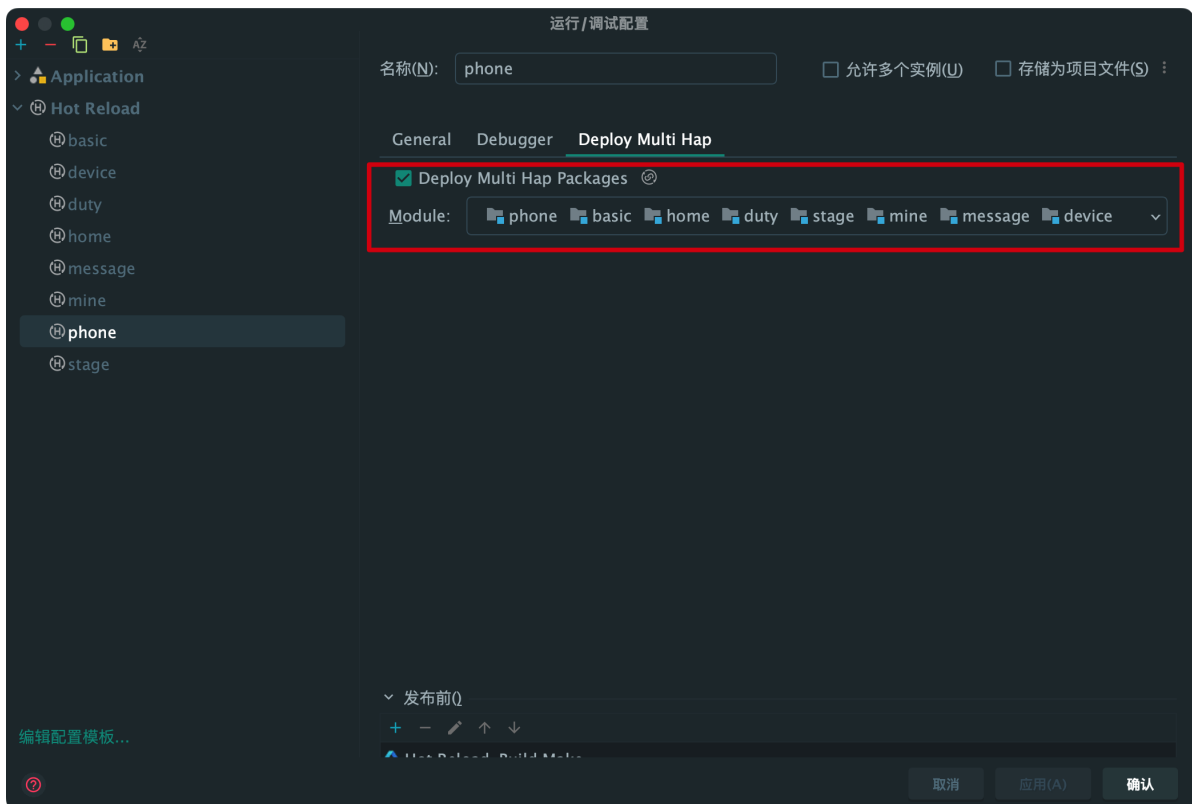
1. 选择编辑配置



2. 安装依赖的动态共享包 (Shared Library)



3. 选择依赖模块



重新运行项目即可

小结:

分层设计的好处?

1. 为华为的手机、2in1等1+8全场景设备提供支持，实现了“一次开发，多端部署”的开发理念
2. **产品定制层、基础特性层和公共能力层**，为开发者构建了一个清晰、高效、可扩展的设计架构
每层模块的作用?
3. common
4. features

5. product

模块资源如何共享?

6. 定义共享资源 (arkts/arkui)

7. 入口导出

8. 使用模块在模块下oh-package.json5 中引入依赖并同步

:::success

说明: 课程使用创建好的项目进行学习开发

:::

二次开发项目

二开项目说明：

1. 基础配置和数据
2. 静态页面资源

1. 基础公共模块：basic
2. 核心业务模块：home、duty、stage、mine、message、device
3. 产品入口模块：phone

1. 入口页面 Index (tab 页签)：首页、负责老人、护理工作台、我的、设备管理（安装、绑定、控制、报警）

项目基础模版：<https://gitee.com/shuiruohanyu/yikang-elder-care>

- 拉取代码

```
1 | $ git clone https://gitee.com/shuiruohanyu/yikang-elder-care
```

Fence 1

:::info

使用DevEcoStudio打开项目

:::

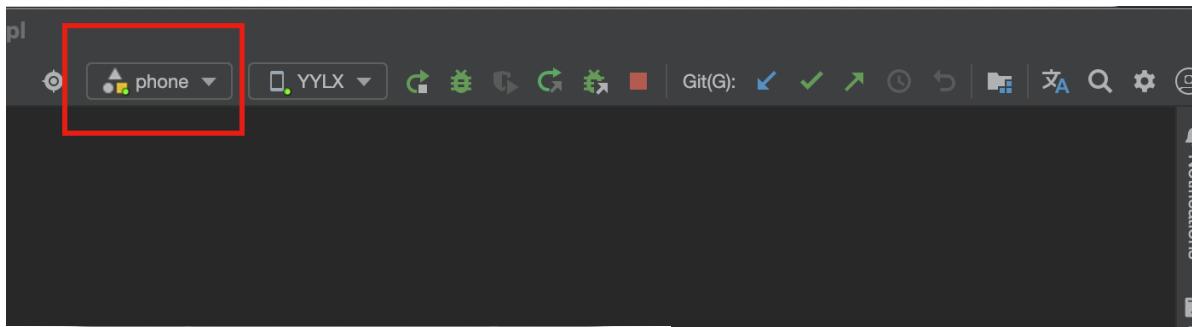
- 修改项目的包名和应用名

```
1 | {
2 |   "app": {
3 | +   "bundleName": "xxxxxxxxx",
4 |     "vendor": "example",
5 |     "versionCode": 1000000,
6 |     "versionName": "1.0.0",
7 |     "icon": "$media:app_icon",
8 |     "label": "$string:app_name"
9 |   }
10 | }
```

Fence 2

```
1 | {
2 |   "string": [
3 |     {
4 |       "name": "module_desc",
5 |       "value": "模块描述"
6 |     },
7 |     {
8 |       "name": "EntryAbility_desc",
9 |       "value": "翼康养老-员工端"
10 |    },
11 |    {
12 |      "name": "EntryAbility_label",
13 | +     "value": "xxxxxx"
14 |    }
15 |  ]
}
```

- 选择phone模块，因为phone模块为hap包，作为设备的入口

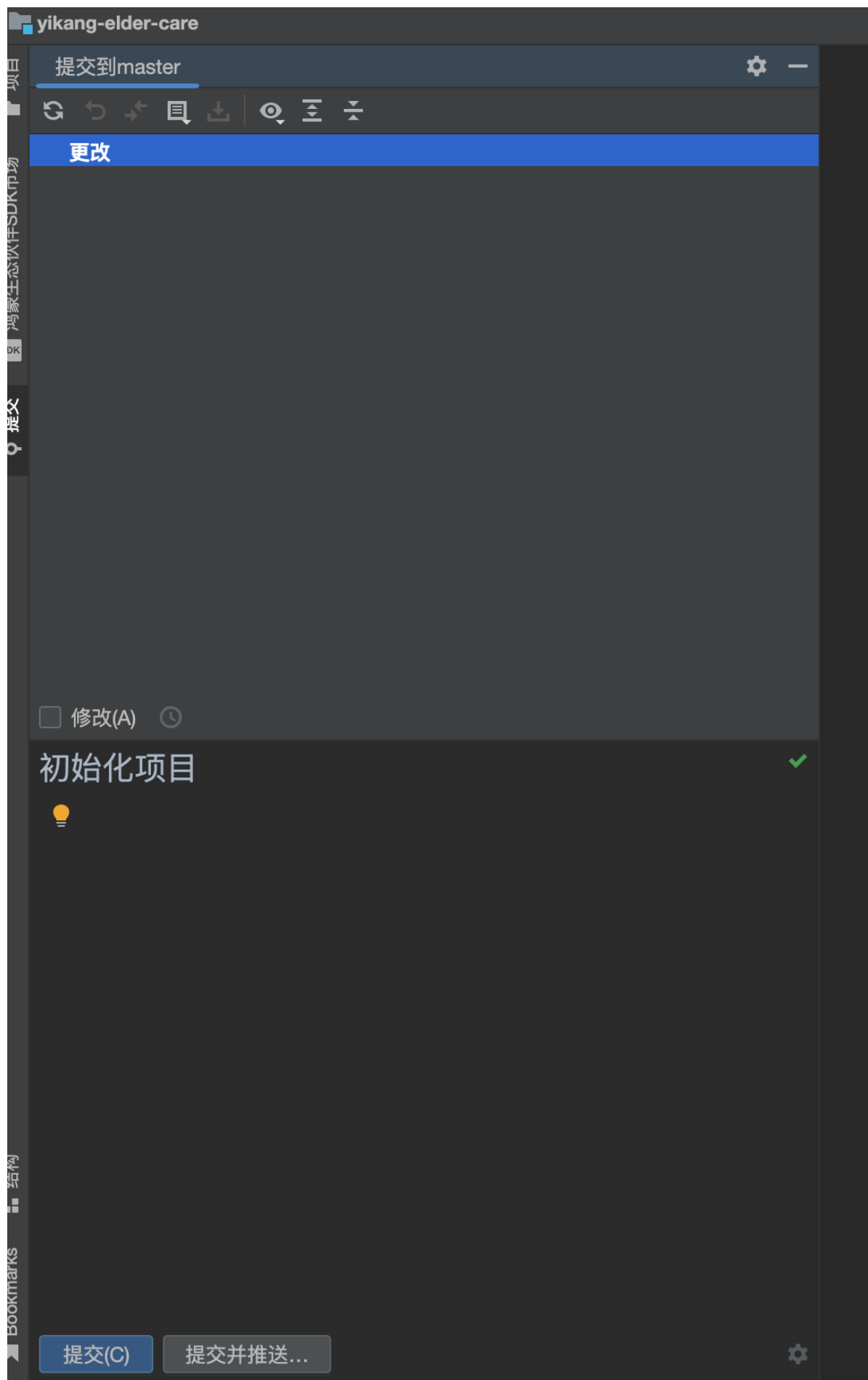


- 在gitee上新建仓库
- 在本地代码进行初始化(先删除本地的.git文件夹-该文件夹是隐藏文件夹)

```
1 | $ git init
```

Fence 4

- 提交代码(使用编辑器DevStudio)



模块和目录结构

模块说明和目录

- common公共业务模块
- features 业务模块: home首页、duty负责老人、stage工作台、mine我的、message消息、device设备
- product入口模块

- 目录规范



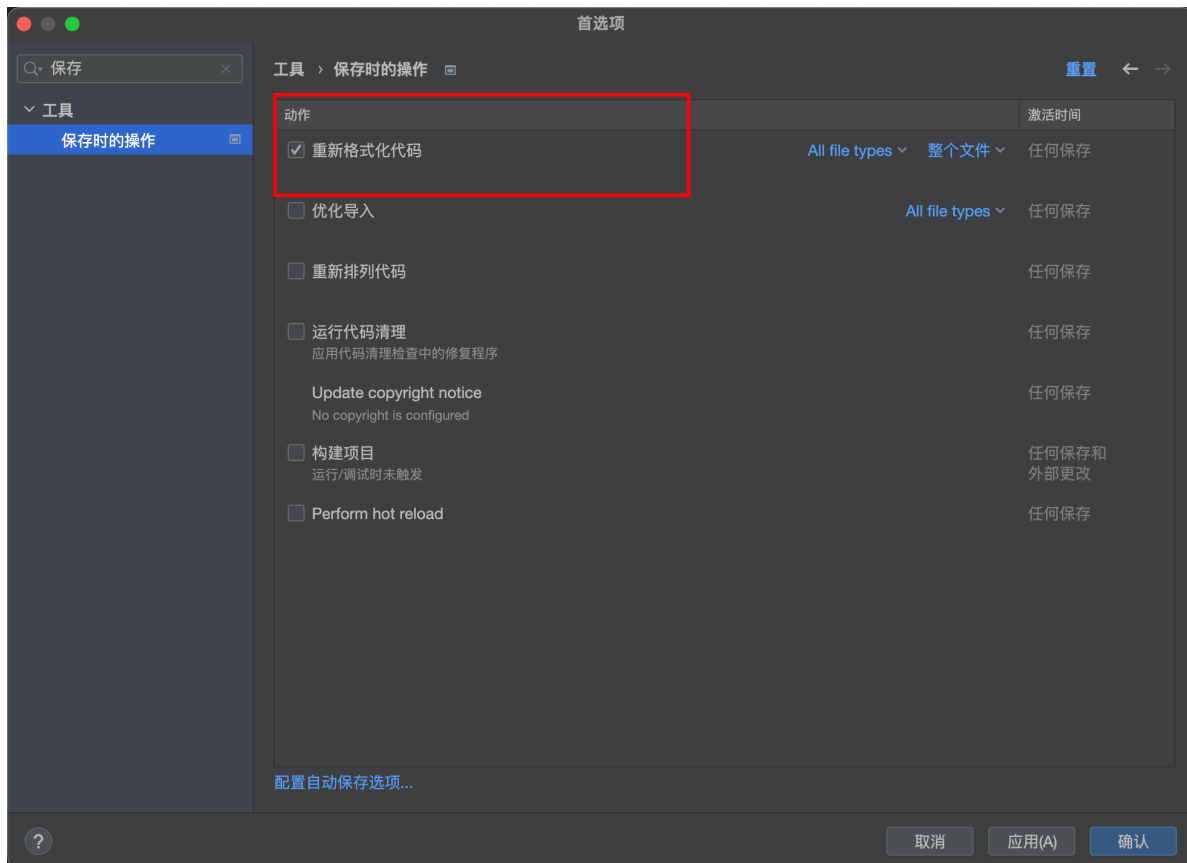
- 依赖关系

模板中已配置好，无需再写

说明：

1. 在模块的 oh-package.json5 文件的 dependencies 配置模块间的依赖关系

```
1  {
2    "license": "",
3    "devDependencies": {},
4    "author": "",
5    "name": "phone",
6    "description": "Please describe the basic information.",
7    "main": "",
8    "version": "1.0.0",
9    "dynamicDependencies": {},
10   "dependencies": {
11     "@hm/basic": "file:../../common/basic",
12     "@hm/public": "file:../../common/public",
13     "@ec/home": "file:../../features/home",
14     "@ec/duty": "file:../../features/duty",
15     "@ec/stage": "file:../../features/stage",
16     "@ec/mine": "file:../../features/mine"
17   }
18 }
```



配置dayjs和日志插件

[OpenHarmony三方库中心仓](#)

模板中已有插件

- @itheima/wing-care
- dayjs
- @abner/log
- @ohos/axios

• 目前请求的几种方式

- http-鸿蒙原生
- axios-第三方和前端一样-量相对最大
- ok-http-第三方和前端一样
- rcp-鸿蒙原生-几乎相当于axios-刚退出没多久!!!!

• 安装依赖的方式

ohpm install xxxx

• 配置日志和dayjs插件

```
1 import { Log } from '@abner/log'
2 import dayjs from 'dayjs';
3 import 'dayjs/locale/zh-cn'
4
5 // 设置语言为中文
6 dayjs.locale('zh-cn');
7
8 Log.init({
9   tag: "yk-yl",
10  domain: 0xEC00,
11  close: false,
12  isHilog: true,
13  showLogLocation: true,
14  logSize: 200
15 })
```

Fence 6

• 在EntryAbility中引入，并替换ability日志打印的方法

```
1 import "../utils/plugin"
2 import { Log } from '@abner/log';
```

Fence 7

• 替换日志打印方法

```
1 import { AbilityConstant, UIAbility, want } from '@kit.AbilityKit';
2 import { window } from '@kit.ArkUI';
3 import "../utils/plugin"
4 import { Log } from '@abner/log';
5
6 export default class EntryAbility extends UIAbility {
```

```

7   onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
8       Log.info('Ability onDestroy');
9   }
10
11  onDestroy(): void {
12      Log.info('Ability onDestroy');
13  }
14
15  onWindowStageCreate(windowStage: window.WindowStage): void {
16      // Main window is created, set main page for this ability
17      Log.info('Ability onWindowStageCreate');
18
19      windowStage.loadContent('pages/Index', (err, data) => {
20          if (err.code) {
21              Log.error('Failed to load the content. Cause: %{public}s' +
JSON.stringify(err) ?? '');
22              return;
23          }
24          Log.info('Succeeded in loading the content. Data: %{public}s' +
JSON.stringify(data) ?? '');
25      });
26  }
27
28  onWindowStageDestroy(): void {
29      // Main window is destroyed, release UI related resources
30      Log.info('Ability onWindowStageDestroy');
31  }
32
33  onForeground(): void {
34      // Ability has brought to foreground
35      Log.info('Ability onForeground');
36  }
37
38  onBackground(): void {
39      // Ability has back to background
40      Log.info('Ability onBackground');
41  }
42 }

```

axios基础封装

基础工具我们可以自己基于http的kit能力进行封装，比如小时达项目，但是企业中也有很大一部分采用第三方插件，比如axios，此次项目我们针对axios进行定制封装

[OpenHarmony三方库中心仓](#)

- 基础封装

```

1 // 导出封装的axios的实例
2 import { Log } from '@abner/log'
3 import axios, { AxiosError, AxiosResponse, InternalAxiosRequestConfig } from
  '@ohos/axios'
4
5 // 创建一个axios的实例
6 const instance = axios.create({
7   baseURL: 'https://zhyl-harmony.itheima.net', // 基础地址
8   http://www.ithiema.net + /a/b/c/d
9   timeout: 20000 // 网络请求超时时间
10 })
11
12 // 两个拦截器
13 instance.interceptors.request.use((config: InternalAxiosRequestConfig) => {
14   // config就是请求的参数 地址 以及一些头信息
15   // 一般在这个位置统一注入token
16   Log.info("1.请求参数: " + JSON.stringify(config))
17   config.headers.Authorization =
18     'eyJhbGciOiJIUzI1NiJ9.eyJjdXJyZW50VXNlciI6IntcInVzZXJuYW1lXCI6XCIZNTQyMzM4NEBxcS5jb2lciixcInBhc3N3b3JkXCI6XCJCiixcIm5pY2t0YW1lXCI6XCXLmiqTnkIb1kZgxxXCisXCjlbWFPpbFwiOlwiMzU0mjmODRACXEuy29tXCIsXCJyZWFSStMtFtZVwiOlwi5oqk55CG5ZGYMVwLFwibW9iaWxlcXI6XCIXODIxMTAyMTg5NWlwlFwic2V4XCIXCiwXCisXCjkZXB0Tm9cIjpcIjEwMDAwMTAwwMTAwwMDAwMFwiLFWicG9zdE5vXCIXCXMdAwMDEwMDUwMDMwMDBcIixcImRhZGF0TGFTDGV0ZVwiOlwiMFWiLFWiYXZhZGFyXCIXCjodHRwcovL3lqeS1vc3MtdmlkZW9zLm9zcylhY2NlbgVyYXRlLmFsaxl1bmNzLmNvbS9ncnp4aHouanBnXCIsXCJpZFwiOjE2NzE0MDMyNTY1MTkwNzgyNzUsXCJjcMvhZGVUaw1lXCI6MTCxNTU2ODM5MDAwMCxcInVwZGF0ZVRpbWVCiJoXNzE1NTY4MzkwMDAwLFIwiy3JlYXRlQnlcIjoXNjcxNDAMjU2NTE5MDC4MTM4fSIsImV4cCI6MTQ2NzU2NTk0Nzd9.8PrMKBNrVVfcfCI4DgICxp6oBQGqy46m3oglfQCiktzw'
19   return config // 返回的这个参数是必须的 axios就是用我们返回的这个config发送请求
20 }, (error: AxiosError) => {
21   // 说明失败了
22   Log.error("3.异常信息: " + JSON.stringify(error))
23   return Promise.reject(error)
24 }) // 注册请求拦截器- 看看你是不是把参数带全了 有没有必要报个错
25
26 // Promise pending fulfilled rejected
27 instance.interceptors.response.use((response: AxiosResponse) => {
28   // reponse是服务器返回给我的结果
29   // axios的默认返回选项是 包了一层data的数据
30   // 判断 是否执行成功
31   if (response.data && response.data.code !== 200) {
32     // 说明执行出现问题 要提示错误 返回promise的异常
33     AlertDialog.show({ message: response.data.msg }) // 提示错误
34     return Promise.reject(new Error(response.data.msg))
35   }
36   return response
37 })

```

```

34     }
35     Log.info("2.响应结果: " + JSON.stringify(response.data))
36     return response.data.data // 直接返回我们需要的数据
37
38 }, (error: AxiosError) => {
39     AlertDialog.show({ message: error.message }) // 弹出错误信息
40     Log.error("3.异常信息: " + JSON.stringify(error))
41     return Promise.reject(error)
42
43 }) // 注册响应拦截器- 数据回来, 有些东西是不是没有必要啊 没有必要 就脱掉 去掉 删掉
44
45 export { instance }

```

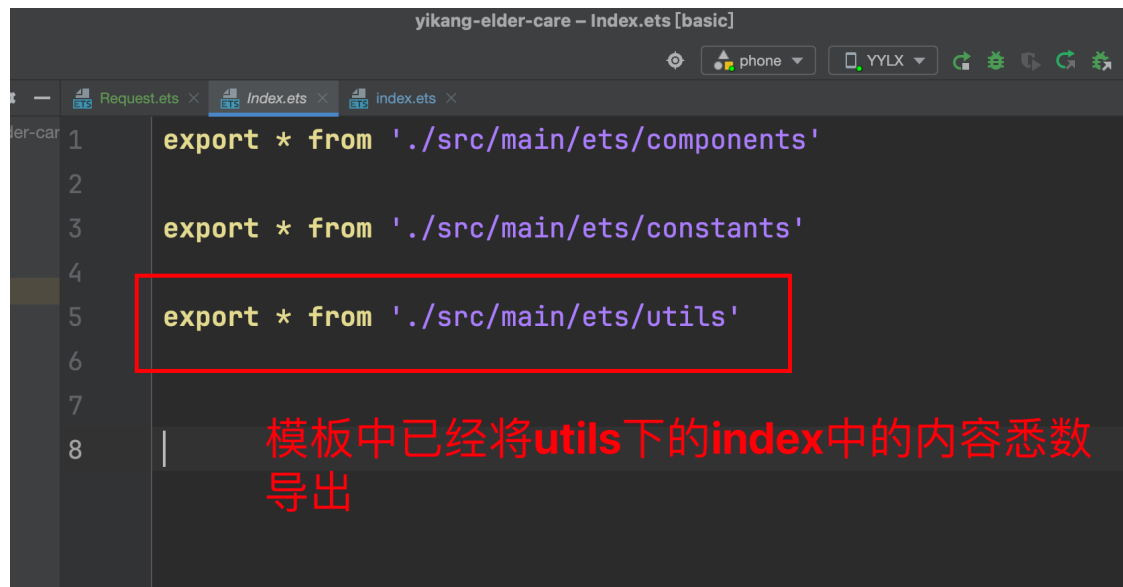
Fence 9

- 在utils/index.ets中导出

```
1 export * from './Request'
```

Fence 10

由于在根目录的Index.ets中, 我们已经将utils整个模块导出, 所以不需要再单独导出



- 在ability中简单测试下

```

1 import { instance } from '@hm/basic'
2 onCreate(want: want, launchParam: AbilityConstant.LaunchParam): void {
3     Log.info('Ability onDestory');
4     instance.get("/employee/user")
5 }

```

Fence 11

Fence 12

看

- 请求成功

axios二次封装制定参数和响应体类型

目前返回的数据结构并没有明确类型指定，无法高效率处理数据，接下来我们对数据结构进行类型指定

- 需求分析

```
// 添加响应拦截器
instance.interceptors.response.use((response: AxiosResponse) => {
  // 对响应数据做点什么
  if (response.data?.code && response.data.code !== 200) {
    AlertDialog.show({
      message: response.data?.msg
    })
    Log.error('4-code请求错误: ' + JSON.stringify(response.data))
    return Promise.reject(response.data)
  }
  Log.warn('2-请求结果: ' + JSON.stringify(response.data))
  // 简化数据返回
  return response.data?.data
}, (error: AxiosError<ErrorRes>) => {
```

如何指定类型?

request<T = any, R = AxiosResponse, D = any>(config: AxiosRequestConfig): Promise

1. 分析request请求泛型参数
2. 知道如何自定义response类型

:::warning

目标：自定义 请求参数 和 响应结果 类型

:::

1. 指定请求参数类型

- 请求参数：request(requestParams:RequestParams)
- 响应结果：request<DataModel = null, DataParams = null>(...)

```
1 import axios, { AxiosRequestConfig, AxiosError, AxiosResponse,
  InternalAxiosRequestConfig, } from '@ohos/axios';
2 // 导出统一的方法使用
3 // R是返回的类型 D是请求类型
4 export function request<Response = null, DataParams = null>(config:
  AxiosRequestConfig) {
5   return instance<null, Response, DataParams>(config)
6 }
```

Fence 13

- 在ability中测试

测试

```
1 import { request } from '@hm/basic'
2
```

```

3 onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
4     Log.info('Ability onDestroy');
5
6     interface UserData {
7         /**
8          * 头像
9          */
10        avatar: null | string;
11
12        /**
13         * 用户id
14         */
15        id: string;
16
17        /**
18         * 用户昵称
19         */
20        nickName: string;
21
22        /**
23         * 用户账号，即邮箱
24         */
25        username: string;
26    }
27
28    request<UserData>({
29        url: '/employee/user'
30    }).then(res => {
31        Log.info("得到数据: " + JSON.stringify(res))
32    })
33 }

```

Fence 14

```

|-----yk-yL-----|
| 得到数据: {"id":"1671403256519078275","username":"35423484@qq.com","nickName":"xiangt",
|  "avatar":"https://yky-t
| each-oss.oss-cn-beijing.aliyuncs.com/ykyL/913d2d5d-7d4a-432a-9af7-7db2561a19af.jpg"}
|

```

删除测试代码，提交代码

Auth鉴权模块

使用首选项管理token的获取和写入

- 创建操作首选项的类并定义token 存储的 key和类型

```
1 import preferences from '@ohos.data.preferences'
2
3 class UserAuthStore {
4     KEY = 'user-store'
5     context?: Context
6
7     // 定义方法
8     // 获取仓库
9     getStore() {
10         return preferences.getPreferences(this.context || getContext(), { name:
'UserStore' })
11     }
12
13     async setUser(data: User) {
14         const store = await this.getStore()
15         await store.put(this.KEY, JSON.stringify(data))
16         await store.flush()
17     }
18
19     async getUser() {
20         const store = await this.getStore()
21         const data = await store.get(this.KEY, '{}') as string
22         return JSON.parse(data) as User
23     }
24
25     async delUser() {
26         const store = await this.getStore()
27         await store.delete(this.KEY)
28         await store.flush()
29     }
30 }
31
32 export class User {
33     token: string = ''
34 }
35
36 export const TokenManager = new UserAuthStore()
```

Fence 15

- utils入口统一导出

```
1 export * from './Auth'
```

Fence 16

提交代码

小结:

用户首选项？用户首选项为应用提供Key-Value键值型的数据处理能力，支持应用持久化轻量级数据，并对其修改和查询。

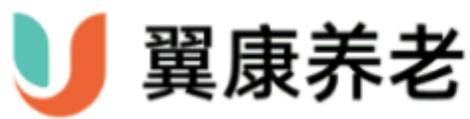
支持的存储的数据类型？数据存储形式为键值对，键的类型为字符串型，值的存储数据类型包括数字型、字符型、布尔型以及这3种类型的数组类型。

广告弹窗创建

- 广告弹窗实现

翼康长相伴
生活更美好





- 修改Ability默认跳转

```
1  async onWindowStageCreate(windowStage: window.WindowStage): Promise<void> {
2      // Main window is created, set main page for this ability
3      Log.info('Ability onWindowStageCreate');
4
5      Log.info('Ability onWindowStageCreate')
6      const win = await windowStage.createSubwindow("ad_win")
7      await win.showWindow()
8
9      win.setUIContent("@bundle:com.heima.elderly.pro/home/ets/pages/Splash/Splash
10 ")
11      windowStage.loadContent('pages/Index', (err, data) => {
12          if (err.code) {
13              Log.error('Failed to load the content. Cause: %{public}s' +
14                  JSON.stringify(err) ?? '');
15              return;
16          }
17          Log.info('Succeeded in loading the content. Data: %{public}s' +
18              JSON.stringify(data) ?? '');
19      });
20  }
```



- 在闪屏广告页进行处理-5秒后销毁窗口

```
1  countdown() {  
2      this.timer = setInterval(() => {  
3          this.count--  
4          if (this.count === 0) {  
5              clearInterval(this.timer)  
6              window.findwindow("ad_win").destroywindow()  
7              return  
8          }  
9      }, 1000)  
10 }
```

小结:

hsp 模块页面跳转? 地址组成: @bundle:包名/模块名/ets/pages/页面名

包名哪里获取? AppScope/app.json5

最佳实践? 统一封装管理路由地址

特性模块路由地址管理

很多页面都是跨包的页面

跨hsp页面跳转 @bundle:包名/模块名称/ets/pages/xxx

- 统一管理 feature 模块页面地址

[HarmonyOS NEXT/MultiFinancialManagement/commons/basic/src/main/ets/constants/RouterConstants.ets · HarmonyOS/Codelabs - Gitee.com](#)

- 将所有地址统一管理在一个静态文件中

```
1 export class RouterConfig {
2     // 基础
3     static BUNDLE: string = 'com.hm.elderly.plus'
4     static readonly SPLASH_URL: string =
5     `@bundle:${RouterConfig.BUNDLE}/home/ets/pages/Splash/Splash`
6     static readonly LOGIN_URL: string =
7     `@bundle:${RouterConfig.BUNDLE}/mine/ets/pages/Login/Login`
8     static readonly USER_SET_URL: string =
9     `@bundle:${RouterConfig.BUNDLE}/mine/ets/pages/Setting/Setting`
10    static readonly USER_NAME_URL: string =
11    `@bundle:${RouterConfig.BUNDLE}/mine/ets/pages/NickName/NickName`
12    // 我的消息
13    static readonly USER_MSG_URL: string =
14    `@bundle:${RouterConfig.BUNDLE}/message/ets/pages/MsgPage`
15    // 负责老人
16    static readonly DUTY_HEALTH_DATA_URL: string =
17    `@bundle:${RouterConfig.BUNDLE}/duty/ets/pages/Health/HealthList`
18    static readonly DUTY_HEALTH_DETAIL_URL: string =
19    `@bundle:${RouterConfig.BUNDLE}/duty/ets/pages/Health/HealthDetail`
20    static readonly DUTY_HEALTH_RECORD_URL: string =
21    `@bundle:${RouterConfig.BUNDLE}/duty/ets/pages/Health/HealthRecord`
22    static readonly DUTY_CARE_TASK_URL: string =
23    `@bundle:${RouterConfig.BUNDLE}/duty/ets/pages/Care/CareTask`
24    // 护理工作台
25    static readonly STAGE_NURSE_RECORDS_URL: string =
26    `@bundle:${RouterConfig.BUNDLE}/stage/ets/pages/Nurse/NurseRecord`
27    static readonly STAGE_EXECUTE_URL: string =
28    `@bundle:${RouterConfig.BUNDLE}/stage/ets/pages/Execute/Execute`
29    // 设备安装
30    static readonly DEVICE_SETUP_URL: string =
31    `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Device/SetupDevice`
32    static readonly DEVICE_SETUP_TIP_URL: string =
33    `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Device/SetupDeviceTip`
34    static readonly DEVICE_ELDER_SEL_URL: string =
35    `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Device/SelectElder`
36    static readonly DEVICE_PRODUCT_SEL_URL: string =
37    `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/device/ProductDevice`
38    // 设备绑定
39    static readonly DEVICE_BIND_URL: string =
40    `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Device/BindDevice`
41    static readonly DEVICE_BIND_TIP_URL: string =
42    `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Device/BindDeviceTip`
```

```

27   static readonly DEVICE_NEAR_URL: string =
   `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Device/NearDevice`
28   static readonly DEVICE_BIND_BLOOD_URL: string =
29     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/BloodManage/BindBlood`
30   // 我的设备
31   static readonly DEVICE_MY_URL: string =
   `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Device/MyDevice`
32   // 血糖
33   static readonly DEVICE_BLOOD_TEST_PREPARE_URL: string =
34
   `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/BloodManage/TestPrepare`
35   static readonly DEVICE_BLOOD_TEST_START_URL: string =
36     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/BloodManage/TestStart`
37   static readonly DEVICE_BLOOD_TEST_RESULT_URL: string =
38     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/BloodManage/TestResult`
39   // 门禁
40   static readonly DEVICE_DOOR_NET_URL: string =
41     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Wifi/NetDoor`
42   static readonly DEVICE_DOOR_NET_CONNECT_URL: string =
43     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Wifi/ConnectNet`
44   static readonly DEVICE_DOOR_SCAN_WIFI_URL: string =
45     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Wifi/Scanwifi`
46   static readonly DEVICE_DOOR_CONNECT_TIP_URL: string =
47     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Wifi/ConnectTip`
48   // 控制
49   static readonly DEVICE_DOOR_CONTROL_URL: string =
50     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Access/DoorControl`
51   static readonly DEVICE_DOOR_BLUE_OPEN_URL: string =
52     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Access/BluetoothOpen`
53   static readonly DEVICE_DOOR_REMOTE_OPEN_URL: string =
54     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Access/RemoteOpen`
55   static readonly DEVICE_DOOR_PASSWORD_URL: string =
56     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Access/TempPassword`
57   static readonly DEVICE_DOOR_OPEN_RECORD_URL: string =
58     `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Access/OpenRecord`
59   // 报警设备
60   static readonly DEVICE_ALARM_URL: string =
   `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Alarm/AlarmDevice`
61   static readonly DEVICE_ALARM_HANDLER_URL: string =
   `@bundle:${RouterConfig.BUNDLE}/device/ets/pages/Alarm/AlarmHandle`
62 }

```

Fence 19

- 在constants/Index.ets中统一导出

```
1 export * from './RouterConfig'
```

Fence 20

2. Ability默认跳转使用

```

1 async onWindowStageCreate(windowStage: window.WindowStage): Promise<void> {
2   // Main window is created, set main page for this ability
3   Log.info('Ability onWindowStageCreate')
4   const win = await windowStage.createSubwindow("ad_win")
5   await win.showWindow()

```

```
6      win.setUIContent(RouterConfig.SPLASH_URL)
7      windowStage.loadContent('pages/Index', (err, data) => {
8          if (err.code) {
9              Log.error('Failed to load the content. Cause: %s' +
JSON.stringify(err) ?? '');
10             return;
11         }
12         Log.info('Succeeded in loading the content. Data: %s' +
JSON.stringify(data) ?? '');
13     });
14 }
```

Fence 21

提交代码

小结:

统一管理的好处? 提高代码可维护性

token判断跳转逻辑

- 判断有无 token 做跳转处理

1. 获取首选项中token 处理跳转

```
1  async onWindowStageCreate(windowStage: window.WindowStage): Promise<void> {
2      // Main window is created, set main page for this ability
3      Log.info('Ability onWindowStageCreate')
4      const win = await windowStage.createSubWindow("ad_win")
5      await win.showWindow()
6      win.setUIContent(RouterConfig.SPLASH_URL)
7      TokenManager.context = this.context // 上下文
8      if ((await TokenManager.getUser()).token) {
9          windowStage.loadContent('pages/Index');
10     } else {
11         windowStage.loadContent(RouterConfig.LOGIN_URL);
12     }
13
14 }
15
```

华为登录介绍

Account Kit（华为帐号服务）提供简单、快速、安全的登录和授权功能，让用户无需输入帐号、密码和繁琐验证，避免因忘记密码而带来的麻烦，为您创建帐号并登录所有HarmonyOS应用和元服务，带来更高的注册转化，同时通过授权获得用户头像昵称、手机号码等信息，提升用户黏性。

文档中心

- 开发前准备

AGC创建项目和应用

- 申请账号授权

ykyl

凭证名称: ykyl (111230731)

凭证类型: 鸿蒙应用

服务: 全部

批量申请

公开权限

敏感权限

☐	权限	权限描述	服务	状态	操作
☐	email	获取您的邮件地址	华为帐号服务	未获取	申请
☐	phone	获取您的手机号	华为帐号服务	已获取	取消
☐	realTimePhone	获取并验证您的手机号	华为帐号服务	未获取	申请
☐	quickLoginMobilePhone	华为帐号一键登录	华为帐号服务	未获取	申请

总计: 4

<1>

10 条/页

跳至

华为提供的面向开发者的应用一站式服务平台

- 配置应用签名证书指纹

1. 在DevEco Studio工具侧[生成密钥 \(.p12\)](#) 和[证书请求文件 \(.csr\)](#)。

注意：保存时填写.p12和.csr 后缀名

Generate Key and CSR

Generate Key

Generate CSR

Key store file (*.p12) *

NEW

CHOOSE EXISTING

Key store password *

?

Key

1.

Create Key Store

Key store file

ids/WingKey/wing-c.p12

Password

Ec123456

Confirm password

Ec123456

CANCEL

OK

Organizational unit

Organization

City or locality

State or province

Country code(XX)

CANCEL

SKIP

NEXT

2.

Generate Key and CSR

Generate Key

Generate CSR

Key store file (*.p12) *

ownloads/WingKey/wing-c.p12

NEW

CHOOSE EXISTING

Key store password *

.....

?

Key

Alias *

meng

?

Password *

.....

?

Validity(years) *

25

^

v

Certificate (At least one of the following cannot be e...

First and last name

wangmeng

Organizational unit

Organization

City or locality

State or province

Country code(XX)

CANCEL

SKIP

NEXT

3.

Generate Key and CSR

Generate Key ———— Generate CSR

Key store file (*.p12) *

Key store password *

Key alias *

Key password *

CSR file (*.csr) *

CANCEL PREVIOUS FINISH

2. 请根据以下使用场景，在[AppGallery Connect](#)侧申请应用证书（.cer）、Profile（.p7b）文件等。

AppGallery Connect

全部服务

解决方案

我的应用

我的项目

应用分析

用户与访问

AppGallery Connect

全部服务

用户与访问

体验演示项目

猫爷王猛

个人信息

服务

已购服务

沙盒测试

测试帐号

开发工具

API Explorer

API错误中心

第三方授权管理

API密钥

Connect API

列表管理

用户列表

证书管理

设备管理

完成证书申请后，您可前往我的项目创建应用并在对应应用项目下[申请profile](#)

证书管理

申请、下载证书

允许申请创建1个发布证书，2个调试证书，具体请参考《[证书管理操作指导](#)》

新增证书

证书名称	证书类型	失效日期	操作
wing-care-release	发布证书	2027-03-04	下载 废除
wing-care-debug	调试证书	2025-03-04	下载 废除
auto_debug_26008600006...	调试证书	2025-03-15	下载 废除

共 3 条10条/页1前往1页

新增证书

* 证书名称:

wing-care-debug15 / 100

* 证书类型:

调试证书

* 选取证书请求文件(CSR):

wing-care.csr

选取

选择第一步生成证书文件

提交

取消

AppGallery Connect

全部服务

我的项目

翼康

H翼康养老

体验演示项目

猫爷王猛

项目设置

HarmonyOS应用

HAP Provision Profile

盈利

应用联运

游戏联运

付费下载

应用内支付服务

管理HAP Provision Profile

添加、删除、下载Profile

允许申请创建100个Profile文件，具体请参考《[Profile管理操作指导](#)》

添加

名称	类型	证书	状态	更新时间	失效日期	操作
wing-care	调试	wing-care-debug	生效	2024-05-08	2025-05-08	删除 下载 查看设备 查看

共 1 条5条/页1前往1页

3. 最终获取的文件


```

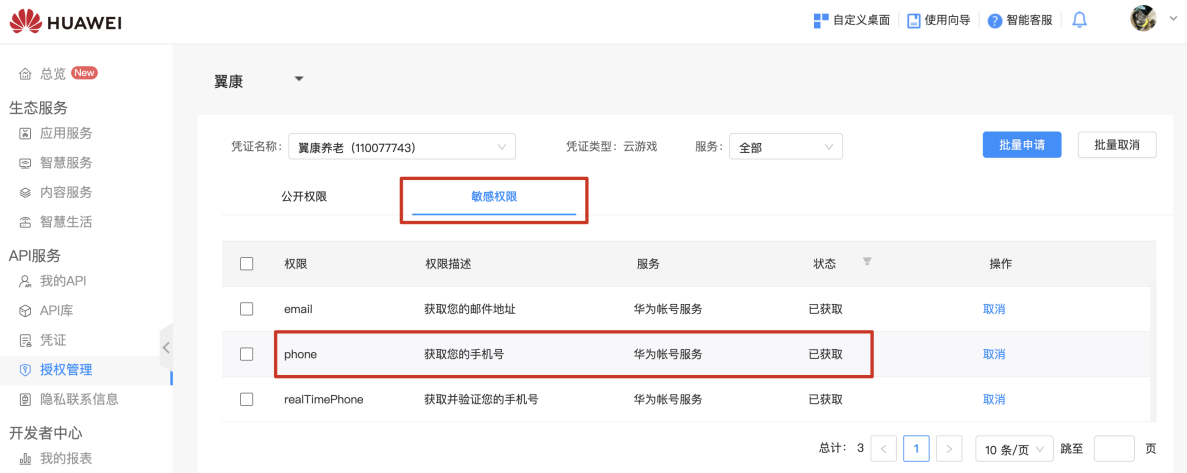
1  {
2    "module": {
3      "name": "phone",
4      "type": "entry",
5      "description": "$string:module_desc",
6      "mainElement": "EntryAbility",
7      "metadata": [
8        {
9 +       "name": "client_id",
10 +      "value": "110077743"
11      }
12    ],
13    "deviceTypes": [
14      "phone",
15      "tablet",
16      "2in1"
17    ],
18    "requestPermissions": [
19      {
20        "name": "ohos.permission.INTERNET"
21      },
22      {
23        "name": "ohos.permission.ACCESS_BLUETOOTH",
24        "reason": "$string:app_name",
25        "usedScene": {
26          "abilities": [ "EntryAbility" ],
27          "when": "inuse"
28        }
29      },
30      {
31        "name": "ohos.permission.GET_WIFI_INFO"
32      },
33      {
34        "name": "ohos.permission.ACCELEROMETER"
35      }
36    ],
37    "deliverywithInstall": true,
38    "installationFree": false,
39    "pages": "$profile:main_pages",
40    "abilities": [
41      {
42        "name": "EntryAbility",
43        "srcEntry": "./ets/entryability/EntryAbility.ets",
44        "description": "$string:EntryAbility_desc",
45        "icon": "$media:icon",
46        "label": "$string:EntryAbility_label",
47        "startWindowIcon": "$media:startIcon",
48        "startWindowBackground": "$color:start_window_background",
49        "exported": true,
50        "skills": [
51          {
52            "entities": [
53              "entity.system.home"
54            ],
55            "actions": [

```

```
56 "action.system.home"
57 ]
58 }
59 ]
60 }
61 ]
62 }
63 }
```

Fence 23

2. 申请手机号授权



开发准备工作?

华为登录实现

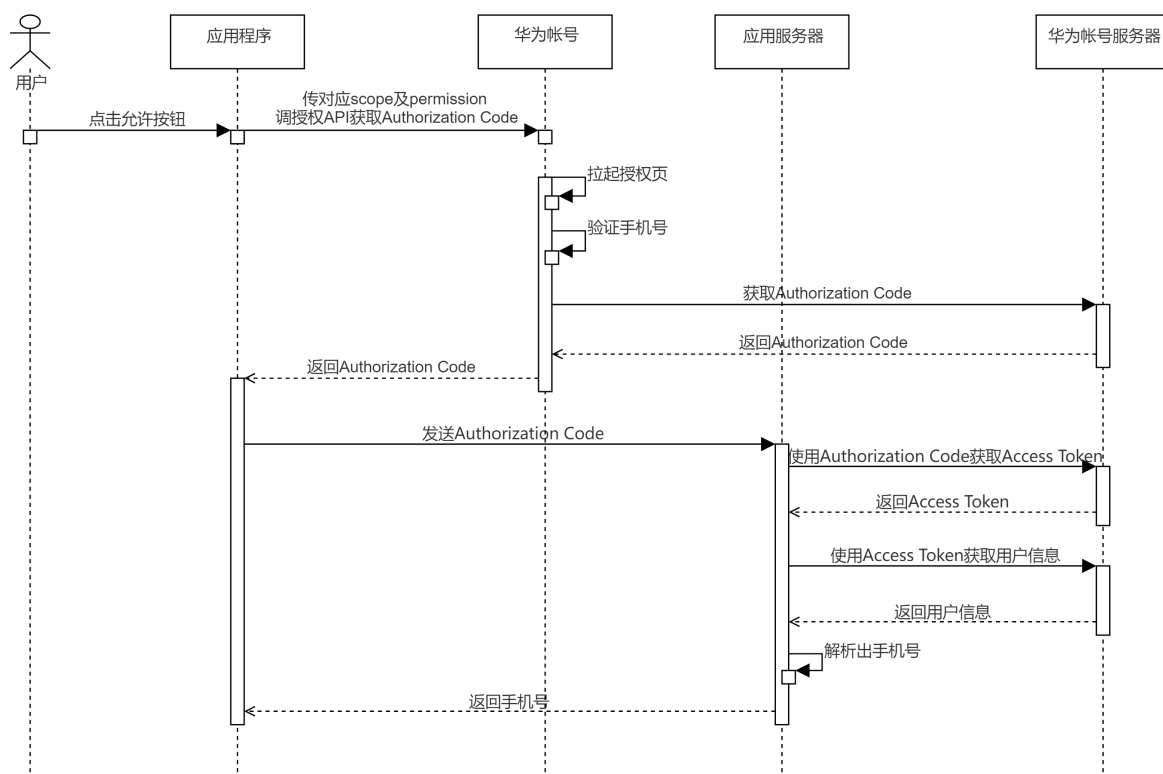
- mine模块



翼康养老



华为登录



场景介绍

Account Kit开放手机号快速验证能力，用于帮助应用向用户发起手机号申请，应用在满足《[常见类型移动互联网应用程序必要个人信息范围规定](#)》（对第三方网站的内容，华为不承担任何责任）中使用手机号的必要业务场景，经用户同意后，应用可获取手机号，为用户提供相应服务。

该能力与[手机号实时验证](#)的区别为：

1. 手机号快速验证，对用户选择的华为帐号绑定的手机号或者新增的手机号进行验证，但不保证是实时验证，首次需要用户授权；
2. 手机号实时验证，对用户发起的每次请求，Account Kit均会对用户选择的手机号或者新增的手机号进行实时验证，以保证手机号的有效性。

说明

应用获取手机号之前，需要完成“获取您的手机号”的scope权限申请，详情参见[scope权限申请手机号](#)，scope审批未完成或未通过，将无法正常使用和获取用户授权手机号。

- 导入[authentication](#)模块及相关公共模块。

```
1 import { authentication } from '@kit.AccountKit';
2 import { util } from '@kit.ArkTS';
```

Fence 24

- 创建登录方法，绑定事件

```
1 // 华为登录
2 login() {
3     // 创建授权请求，并设置参数
4     let authRequest = new
authentication.HuaweiIDProvider().createAuthorizationWithHuaweiIDRequest();
5     // 获取手机号需要传如下scope，传参数之前需要先申请对应scope权限，才能返回对应数据
6     authRequest.scopes = ['phone'];
7     // 获取code需传如下permission
8     authRequest.permissions = ['serviceauthcode'];
9     // 用户是否需要登录授权，该值为true且用户未登录或未授权时，会拉起用户登录或授权页面
10    authRequest.forceAuthorization = true;
11    // 用于防跨站点请求伪造，非空字符串即可
12    authRequest.state = util.generateRandomUUID();
13 }
```

Fence 25

- 调用[AuthenticationController](#)对象的[executeRequest](#)方法执行授权请求，并在Callback中处理授权结果，从授权结果中解析出Authorization Code。

```
1 // 华为登录
2 async login() {
3     // 创建授权请求，并设置参数
4     let authRequest = new
authentication.HuaweiIDProvider().createAuthorizationWithHuaweiIDRequest();
5     // 获取手机号需要传如下scope，传参数之前需要先申请对应scope权限，才能返回对应数据
6     authRequest.scopes = ['phone'];
7     // 获取code需传如下permission
8     authRequest.permissions = ['serviceauthcode'];
9     // 用户是否需要登录授权，该值为true且用户未登录或未授权时，会拉起用户登录或授权页面
10    authRequest.forceAuthorization = true;
```

```

11 // 用于防跨站点请求伪造，非空字符串即可
12 authRequest.state = util.generateRandomUUID();
13 try {
14     let controller = new
authentication.AuthenticationController(getContext(this));
15     const result = await controller.executeRequest(authRequest) as
authentication.AuthorizationWithHuaweiIDResponse;
16     let state = result.state;
17     if (state && authRequest.state != state) {
18         return promptAction.showToast({ message: JSON.stringify(result) });
19     }
20     let authorizationWithHuaweiIDCredential = result.data!;
21     let code = authorizationWithHuaweiIDCredential.authorizationCode;
22     // 开发者处理code
23
24 } catch (error) {
25     promptAction.showToast({ message: error.message });
26
27 }
28 }

```

Fence 26

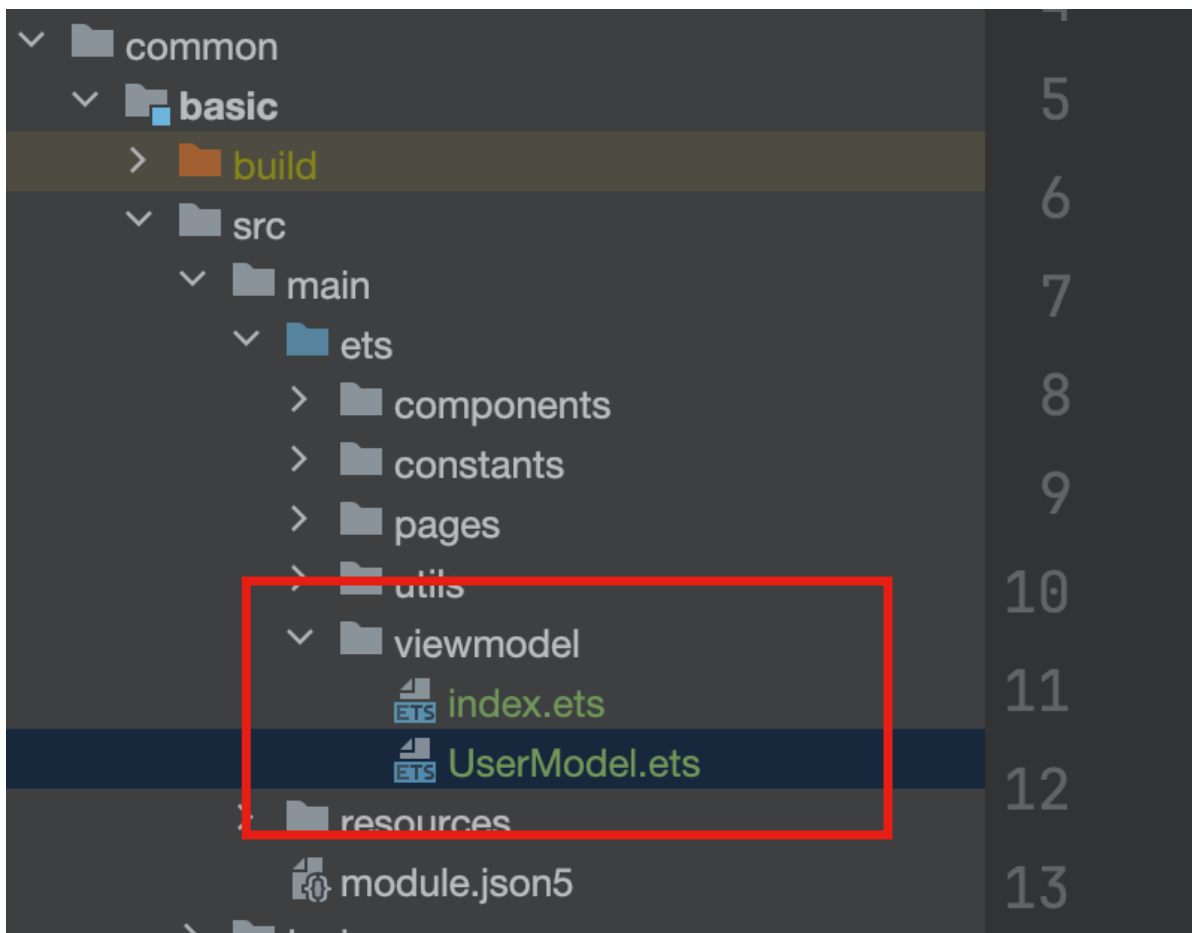
服务端开发

后台配合最终完成开发

注意：华为认证需要后台配合华为认证登录完成手机号检查和注册和返回登录结构

- 封装登录 api 请求函数

统一在 **common/basic** 全局封装



- 定义用户的类型

```
1 // 登录: 参数
2 export interface LoginParams {
3     /**
4      * 华为应用id
5      */
6     clientId?: null | string;
7
8     /**
9      * 华为应用密钥
10     */
11     clientSecret?: null | string;
12
13     /**
14      * 华为账号授权码
15      */
16     code: string;
17 }
```

Fence 27

- 在viewmodel/index.ets中导出

```
1 export * from './UserModel'
```

Fence 28

- 在basic模块封装用户API

```
1 import { LoginParams } from '../viewmodel'
2 import { request } from '../utils'
3
4 /**
5  * 华为手机号授权登录
6  * @param data
7  * @returns
8  */
9 export function loginApi(data: LoginParams): Promise<string> {
10     return request<string, LoginParams>({ url: '/employee/login', method:
11         'post', data })
12 }
```

Fence 29

- 在api/index.ets导出

```
1 export * from './UserApi'
```

Fence 30

- 在basic/Index.ets中导出

```
1 export * from './src/main/ets/api'
2
3 export * from './src/main/ets/viewmodel'
```

Fence 31

- mine模块调用登录接口并存储token

```

1  async login() {
2      // 创建授权请求，并设置参数
3      let authRequest = new
authentication.HuaweiIDProvider().createAuthorizationWithHuaweiIDRequest();
4      // 获取手机号需要传如下scope，传参数之前需要先申请对应scope权限，才能返回对应数据
5      authRequest.scopes = ['phone']; // 快速登录手机号
6      // 获取code需传如下permission
7      authRequest.permissions = ['serviceauthcode'];
8      // 用户是否需要登录授权，该值为true且用户未登录或未授权时，会拉起用户登录或授权页面
9      authRequest.forceAuthorization = true;
10     // 用于防跨站点请求伪造，非空字符串即可
11     authRequest.state = util.generateRandomUUID();
12     try {
13         let controller = new
authentication.AuthenticationController(getContext(this)); // 创建拉起授权请求
的controller
14         const result = await controller.executeRequest(authRequest) as
authentication.AuthorizationWithHuaweiIDResponse;
15         if (authRequest.state !== result.state) {
16             return promptAction.showToast({ message: '华为认证登录失败' })
17         }
18         const code = result.data?.authorizationCode! // 拿到真正的code
19         const token = await loginApi({
20             code,
21             clientId: '111230731', // 实际情况决定不能写
22             clientSecret:
'44c2ecbcc275ef10d1a7c6348d8a4695dc53f1f7ff87677c92ecdc29564ca93d' // 实际情
况下决定不能写
23         })
24         // 写入首选项
25         await TokenManager.setUser({ token })
26         promptAction.showToast({ message: '登录成功' })
27         router.replaceUrl({
28             url: "pages/Index" // 跳转主页
29         })
30
31     } catch (error) {
32         promptAction.showToast({ message: error.message })
33     }
34
35
36 }
37

```

Fence 32

- 测试

登录成功后，杀掉应用，再次进入闪屏后直接跳转首页

坑点：目前API11的项目才可以使用华为登录认证，API12的项目会提示指纹证书不正确，切记！！

小结：

手机号快速登录流程？

basic模块

- ```

1 // 添加请求拦截器
2 instance.interceptors.request.use(async (config: InternalAxiosRequestConfig)
=> {
3 // 对请求数据做点什么
4 // config.headers.Authorization =
5 //
6 'eyJhbGciOiJIUzI1NiJ9.eyJjdXJyZW50VXNlciI6IntcInVzZXJ1eWw1lXCI6XCIZNTQyMzQ4NE
BxcS5jb2l2cixcInBhc3N3b3JkXCI6XCJcIixcIm5pY2t0YWw1lXCI6XCLmIqTnkIb1kZgxcXCIsXC
JlbWVpbFwiO1wiMzU0MjM0ODRACXEuY29tXCIsXCJyZWFSMmFtZVwiO1wi5oqk55CG5ZGYMVwiLF
wibW9iaWx1XCI6XCIXODIXMTAYMTg5NlwiLFwic2V4XCI6XCIXXCIsXCJkZXB0Tm9cIjpcIjEwMD
AwMTAwNTAwMDAwMFwiLFwicG9zdE5vXCI6XCIXMDAwMDEwMDUwMDMwMDBCiixcImRhdGFTdGF0ZV
wiO1wiMFWiLFwiYXZhdGFyXCI6XCJodHRwczovL3lqeS1vc3Mtdm1kZW9zLm9zcy1hY2N1bGVyYX
RlLmFsaX1lbmNzLmNvbS9ncnp4aHouanBnXCIsXCJpZFWiOjE2NzE0MDMyNTY1MTkwNzgyNzUsXC
JjcmVhdGVUaw1lXCI6MTCxNTU2ODM5MDAwMCxcInVwZGF0ZVRpbWVcIjoxNzE1NTY4MzkwMDAwLF
wiY3JlYXRlQnlcIjoxNjcxNDZmZjU2NTE5MDE4MTM4fSIsImV4cCI6MTQ2NzU2NTk0Nzd9.8PrMK
BNRVVCfcI4DgICxp6oBQGqy46m3og1FQCiktzw'
7 Log.info(`1-请求配置${config.method!}-${config.url}`:
8 ${JSON.stringify(config.params || config.data)}`)
9 const userToken = await TokenManager.getUser()
10 if (userToken.token) {
11 config.headers.Authorization = userToken.token
12 }
13 return config;
14 }, (error: AxiosError) => {
15 // 对请求错误做些什么
16 return Promise.reject(error);
17 });

```

- 响应拦截器, 处理 401

```

1 // Promise pending fullfilled rejected
2 instance.interceptors.response.use((response: AxiosResponse) => {
3 // response是服务器返回给我的结果
4 // axios的默认返回选项是 包了一层data的数据
5 // 判断 是否执行成功
6 if (response.data && response.data.code !== 200) {
7 // 说明执行出现问题 要提示错误 返回promise的异常
8 AlertDialog.show({ message: response.data.msg }) // 提示错误
9 return Promise.reject(new Error(response.data.msg))
10 }
11 Log.info("2.响应结果: " + JSON.stringify(response.data))
12 return response.data.data // 直接返回我们需要的数据
13
14 }, (error: AxiosError) => {

```

```
15 // 4xx 5xx会走到axios的响应拦截器的错误回调函数中
16 if (error.response?.status === 401) {
17 // 说明登录失效了 因为有效期为180天 没必要重新用复活 直接重新登录即可
18 TokenManager.delUser() // 删除原有token
19 router.replaceUrl({
20 url: RouterConfig.LOGIN_URL
21 })
22 } else {
23 // 除401之外的错误可以弹出显示
24 AlertDialog.show({ message: error.message }) // 弹出错误信息
25 }
26 //
27 Log.error("3.异常信息: " + JSON.stringify(error))
28 return Promise.reject(error)
29
30 }) // 注册响应拦截器- 数据回来, 有些东西是不是没有必要啊 没有必要 就脱掉 去掉 删掉
```

# 设备管理builder抽提

设备管理结构复用

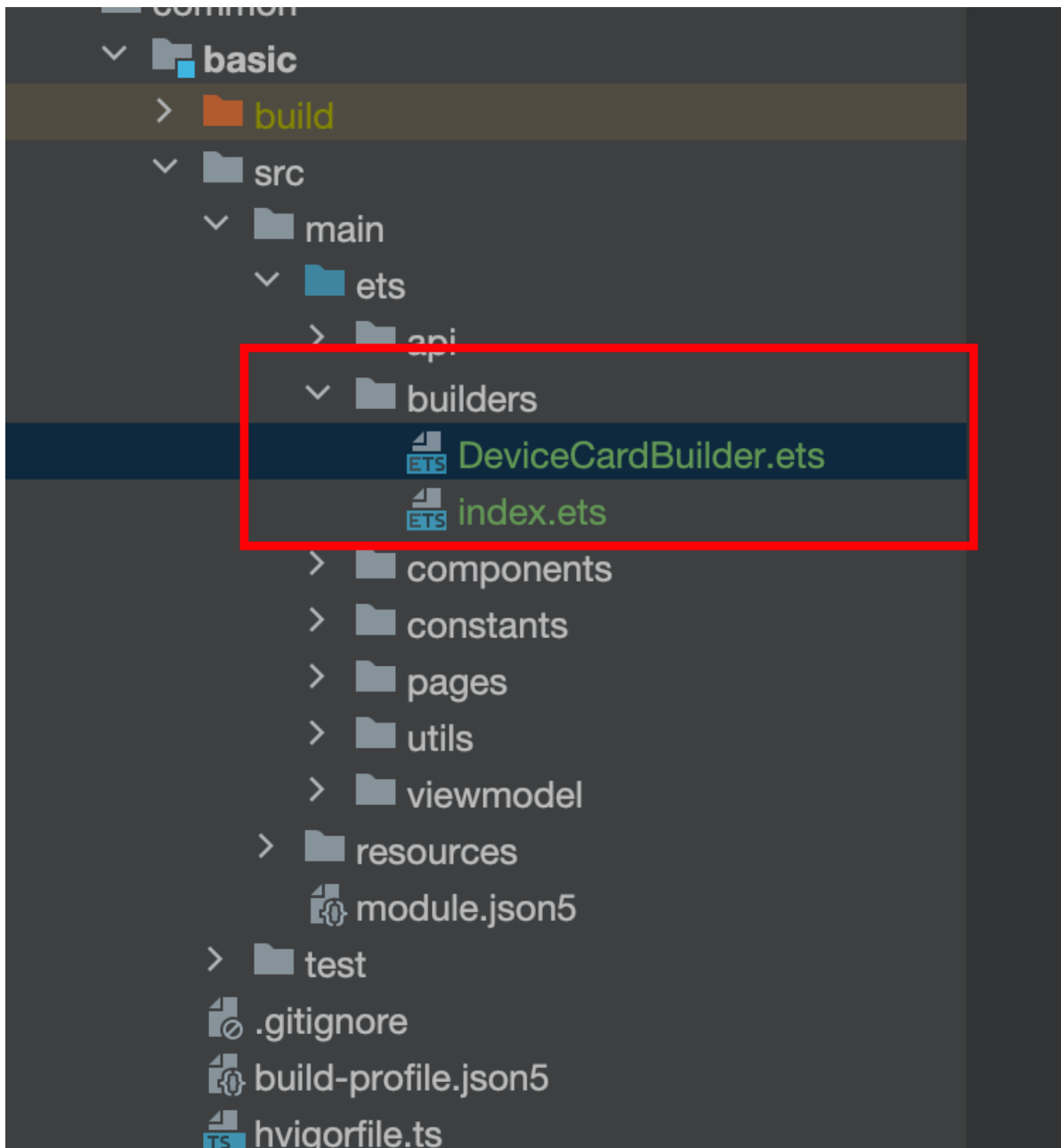
- 复用首页设备管理样式



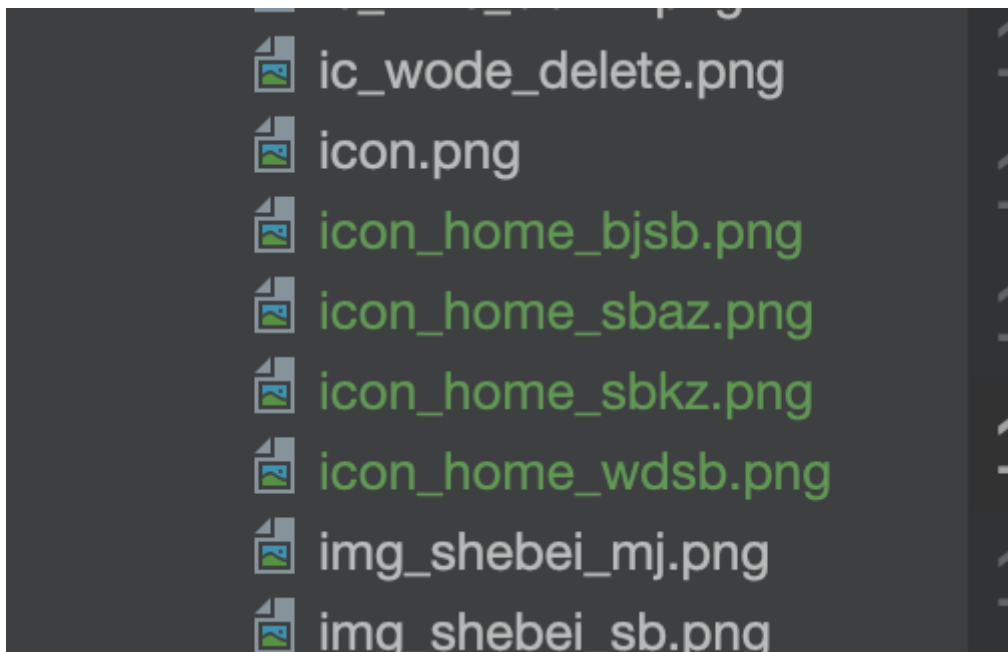
我的页签中同样也需要这样四个的卡片，将上个小节封装的好的内容提到basic中，这样不论是mine模块还是

我的页面都可以非常轻松的实现同样的效果

- 将features中Home的设备管理的样式抽提到basic中的builders目录下



- 拷贝图片资源



- 抽提公共的builder

```

1 import { ResManager } from '../utils'
2 import { router } from '@kit.ArkUI'
3 import { RouterConfig } from '../constants'
4
5 class CareBuilderParams {
6 title: string = ""
7 icon: ResourceStr = ""
8 direction: "horizontal" | "vertical" = "horizontal" // 排列方式
9 pageUrl: string = "" // 跳转地址
10 }
11
12 // 模块item
13 @Builder
14 /**
15 *
16 * @param title
17 * @param icon
18 * @param pos
19 * @param url
20 */
21 function careItem(item: CareBuilderParams) {
22 if (item.direction === "horizontal") {
23 // 护理工作台
24 Row({ space: 10 }) {
25 Image(item.icon)
26 .width(48)
27 .aspectRatio(1)
28 Text(item.title)
29 .fontColor(ResManager.EC_MID_19)
30 .fontSize(ResManager.EC_NORMAL_FS)
31 }
32 .padding({
33 top: 20,
34 bottom: 23,
35 left: 13,
36 right: 13
37 })
38 // .borderRadius(8)
39 // .layoutWeight(1)
40 // .backgroundColor(ResManager.EC_MID_WHITE)
41 .modStyle()
42 .onClick(() => {
43 item.pageUrl && router.pushUrl({
44 url: item.pageUrl
45 })
46 })
47 } else {
48 // 设备管理
49 Column({ space: 10 }) {
50 Image(item.icon)
51 .width(40)
52 .aspectRatio(1)
53 Text(item.title)
54 .fontColor(ResManager.EC_MID_19)
55 .fontSize(ResManager.EC_SMALL_FS)

```

```

56 }
57 .padding({ top: 23, bottom: 23 })
58 // .borderRadius(8)
59 // .layoutWeight(1)
60 // .backgroundColor(ResManager.EC_MID_WHITE)
61 .modStyle()
62 .onClick(() => {
63 item.pageUrl && router.pushUrl({
64 url: item.pageUrl
65 })
66 })
67 }
68 }
69
70 // 可复用样式
71 @Styles
72 function modStyle() {
73 .borderRadius(8)
74 .layoutWeight(1)
75 .backgroundColor(ResManager.EC_MID_WHITE)
76 }
77
78 @Builder
79 export function DeviceBuilder() {
80 Row({ space: 10 }) {
81 careItem({
82 title: '设备安装',
83 icon: $r('app.media.icon_home_sbaz'),
84 direction: 'vertical',
85 pageUrl: RouterConfig.DEVICE_SETUP_URL
86 })
87 careItem({
88 title: '设备绑定',
89 icon: $r('app.media.icon_home_sbkz'),
90 direction: 'vertical',
91 pageUrl: RouterConfig.DEVICE_BIND_URL
92 })
93 careItem({
94 title: '我的设备',
95 icon: $r('app.media.icon_home_wdsb'),
96 direction: 'vertical',
97 pageUrl: RouterConfig.DEVICE_MY_URL
98 })
99 careItem({
100 title: '报警设备',
101 icon: $r('app.media.icon_home_bjsb'),
102 direction: 'vertical',
103 pageUrl: RouterConfig.DEVICE_ALARM_URL
104 })
105 }
106 .width('100%')
107 .justifyContent(FlexAlign.SpaceBetween)
108 }
109
110 @Builder

```

```

111 export function CareBuilder() {
112 Row({ space: 10 }) {
113 careItem({
114 title: '护理任务',
115 icon: $r('app.media.icon_home_hlrw'),
116 direction: 'horizontal',
117 pageUrl: ""
118 })
119 careItem({
120 title: '护理记录',
121 icon: $r('app.media.icon_home_hlj1'),
122 direction: 'horizontal',
123 pageUrl: ""
124 })
125 }
126 .width('100%')
127 .justifyContent(FlexAlign.SpaceBetween)
128 }

```

Fence 35

- 在basic/builders/index.ets导出

```

1 export * from './DeviceCardBuilder'

```

Fence 36

- 统一在basic/Index.ets中导出

```

1 export * from './src/main/ets/builders'

```

Fence 37

- 在Home组件中应用

```

// 1.4 设备管理
Column() {
 // 标题
 Text('设备管理')
 .fontColor(ResManager.EC_MID_19)
 .fontSize(ResManager.EC_MODULE_TITLE_FS)
 .fontWeight(500)
 .margin({ bottom: 13 })

 // 内容
 DeviceBuilder()
}
.width('100%')
.alignItems(HorizontalAlign.Start)
}

```

- 在Mine组件中应用

```
// 2. 快捷导航1
Column({ space: 10 }) {
 Row({ space: 5 }) {
 Text('设备管理')
 .fontColor(ResManager.EC_MID_19)
 .fontSize(ResManager.EC_MODULE_TITLE_FS)
 .fontWeight(500)
 }
 .width('100%')

 // 导航: 设备管理
 DeviceBuilder()
}

.margin({ top: -80, left: ResManager.EC_CONTENT_SPACE, right: ResManager.EC_CON
// .width('100%')
```

提交代码

# 用户信息-获取

---

- 获取登录护理员信息



刘大壮

sijigaowa@163.com

## 设备管理



设备绑定



设备安装



我的设备



报警设备

我的消息



退出

## basic模块

- 定义用户数据类型

```
1 // 我的：登录用户
2 export interface UserData {
3 /** 头像 */
4 avatar: null | string;
5
6 /** 用户id */
7 id: string;
8
9 /** 用户昵称 */
10 nickName: string;
11
12 /** 用户账号，即邮箱 */
13 username: string;
14 }
15 export class UserDataModel implements UserData {
16 avatar: null | string = null
17 id: string = ''
18 nickName: string = ''
19 username: string = ''
20
21 constructor(model: UserData) {
22 this.avatar = model.avatar
23 this.id = model.id
24 this.nickName = model.nickName
25 this.username = model.username
26 }
27 }
28
```

Fence 38

- 封装 api 函数

```
1 /**
2 * 获取登录用户信息
3 * @returns
4 */
5 export function getUserApi(): Promise<UserDataModel> {
6 return request<UserDataModel>({ url: '/employee/user' })
7 }
```

Fence 39

## 在mine模块中获取信息

- 声明响应式数据
- 初始化获取数据
- 更新UI显示

```

1 @State
2 user: UserDataModel = new UserDataModel({} as UserData)
3
4 async getUser() {
5 const data = await getUserApi()
6 this.user = data
7 }
8
9 aboutToAppear(): void {
10 this.getUser()
11 }

```

Fence 40

```

1 Column({ space: 5 }) {
2 Image(this.user.avatar ||
3
4 'https://img2.baidu.com/it/u=130573784,2479509663&fm=253&fmt=auto&app=138&f=
JPEG?w=500&h=534')
5 .width(74)
6 .aspectRatio(1)
7 .borderRadius(37)
8 .border({ width: 5, color: '#AEE8E4' })
9 .shadow({
10 offsetX: 0,
11 offsetY: 2,
12 radius: 21,
13 color: '#0aa9941f'
14 })
15 Text(this.user.nickName || "-")
16 .fontSize(ResManager.EC_PAGE_TITLE_FS)
17 .fontColor(ResManager.EC_MID_BLACK)
18 .fontWeight(500)
19 Text(this.user.username || "-")
20 .fontSize(13)
21 .fontColor(ResManager.EC_MID_59)
22 }
23 .margin({ top: 64 - this.topHeight })

```

Fence 41

- 点击个人信息区域跳转设置详情页

```

1 Column({ space: 5 }) {
2 Image(this.user.avatar ||
3
4 'https://img2.baidu.com/it/u=130573784,2479509663&fm=253&fmt=auto&app=138&f=
JPEG?w=500&h=534')
5 .width(74)
6 .aspectRatio(1)
7 .borderRadius(37)
8 .border({ width: 5, color: '#AEE8E4' })
9 .shadow({
10 offsetX: 0,
11 offsetY: 2,

```

```
11 radius: 21,
12 color: '#0aa9941f'
13 })
14 Text(this.user.nickName || "-")
15 .fontSize(ResManager.EC_PAGE_TITLE_FS)
16 .fontColor(ResManager.EC_MID_BLACK)
17 .fontWeight(500)
18 Text(this.user.username || "-")
19 .fontSize(13)
20 .fontColor(ResManager.EC_MID_59)
21
22 }
23 .margin({ top: 64 - this.topHeight })
24 .onClick(() => {
25 router.pushUrl({
26 url: RouterConfig.USER_SET_URL
27 })
28 })
```

Fence 42

提交代码

# 用户信息-编辑器页获取信息

- 修改昵称
- 上传修改头像

## ← 个人设置

头像



昵称

护理组大姐2 >

首先获取用户的信息

- 获取用户信息（和我的组件一致）

```
1 @State
2 user: UserDataModel = new UserDataModel({} as UserData)
3
4 async getUser() {
5 this.user = await getUserApi()
6 }
7
8 aboutToAppear(): void {
9 this.getUser()
10 }
11
```

Fence 43

还可以在跳转的时候通过传递参数的形式将整个用户对象传递过去

如

```
1 router.pushUrl({
2 url: RouterConfig.USER_SET_URL,
3 params: this.user
4 })
```

```

1 @State
2 user: UserDataModel = new UserDataModel({} as UserData)
3
4 aboutToAppear(): void {
5 const params = router.getParams() as UserDataModel
6 if (params) {
7 // 一定得判断 因为鸿蒙里面没有传值的情况下 params是个undefined
8 this.user = params
9 }
10 }

```

Fence 45

- 定义一个枚举类型

```

1 enum ShowType {
2 NICKNAME,
3 AVATAR
4 }
5

```

Fence 46

- 更新UI

```

1 @Builder
2 cellItem(title: string, type: ShowType) {
3 Row() {
4 Text(title)
5 .fontColor(ResManager.EC_MID_19)
6 .fontSize(ResManager.EC_MODULE_TITLE_FS)
7 .fontWeight(500)
8 Row({ space: 10 }) {
9 if (type === ShowType.NICKNAME) {
10 // 昵称修改
11 Text(this.user.nickName || "-")
12 .fontColor(ResManager.EC_MID_59)
13 .fontSize(ResManager.EC_NORMAL_FS)
14 .onClick(() => {
15 router.pushUrl({
16 url: 'pages/mine/NickNamePage'
17 })
18 })
19 } else if (type === ShowType.AVATAR) {
20 // 上传
21 Image(this.user.avatar ||
22
23 'https://img2.baidu.com/it/u=130573784,2479509663&fm=253&fmt=auto&app=138&f=
JPEG?w=500&h=534')
24 .width(34)
25 .aspectRatio(1)
26 .borderRadius(17)
27 .onClick(() => {
28 this.uploadDialog.open()
29 })
30 }
31 }
32 }
33 }

```

```

29 }
30 Image($r('app.media.ic_arrow_black'))
31 .width(7)
32 .height(16)
33 }
34 }
35 .width('100%')
36 .padding({ top: 14, bottom: 14 })
37 .justifyContent(FlexAlign.SpaceBetween)
38 }

```

Fence 47

- 修改调用的位置

```

1 ListItem() {
2 this.cellItem('头像', ShowType.NICKNAME)
3 }
4
5 ListItem() {
6 this.cellItem('昵称', ShowType.AVATAR)
7 }

```

Fence 48

# 唤起拍照-唤起相册-取消弹窗

上传组件已经在模板中**basic**模块准备好，我们需要进一步实现上传功能

组件位置：**common/basic/src/main/ets/components/EcDialogUpload.ets**

## 1. 点击拍照调用 `cameraPicker.pick`

```
1 ListItem() {
2 Text('拍照')
3 .fontColor(ResManager.EC_MAIN_COLOR)
4 .fontSize(ResManager.EC_MODULE_TITLE_FS)
5 .fontWeight(500)
6 .width('100%')
7 .textAlign(TextAlign.Center)
8 .padding(15)
9 }
10 .onClick() => {
11 this.takePhoto()
12 })
13
```

Fence 49

```
1 // 拍照
2 async takePhoto() {
3 let mContext = getContext(this) as common.Context;
4 try {
5 let pickerProfile: cameraPicker.PickerProfile = {
6 cameraPosition: camera.CameraPosition.CAMERA_POSITION_BACK
7 };
8 let pickerResult: cameraPicker.PickerResult = await
cameraPicker.pick(mContext,
9 [cameraPicker.PickerMediaType.PHOTO,
cameraPicker.PickerMediaType.VIDEO], pickerProfile);
10 console.log("the pick pickerResult is:" +
JSON.stringify(pickerResult));
11 Log.info(pickerResult.resultUri)
12 } catch (error) {
13 let err = error as BusinessError;
14 console.error(`the pick call failed. error code: ${err.code}`);
15 }
16 }
17
```

Fence 50

- 点击相册选择调用 `picker.PhotoViewPicker`

```

1 ListItem() {
2 Text('从相册选择')
3 .fontColor(ResManager.EC_MAIN_COLOR)
4 .fontSize(ResManager.EC_MODULE_TITLE_FS)
5 .fontWeight(500)
6 .width('100%')
7 .textAlign(TextAlign.Center)
8 .padding(15)
9 }
10 .onClick(() => {
11 this.selectPhoto()
12 })

```

Fence 51

- 实现选择相册方法

```

1 // 选择相册
2 async selectPhoto() {
3 // 选择文件
4 const photoSelectOptions = new picker.PhotoSelectOptions()
5 photoSelectOptions.MIMETYPE = picker.PhotoViewMIMETypes.IMAGE_TYPE
6 photoSelectOptions.maxSelectNumber = 1
7
8 const photoViewPicker = new picker.PhotoViewPicker()
9 const result = await photoViewPicker.select(photoSelectOptions)
10 // 拿到照片
11
12 }

```

Fence 52

- 点击取消关闭弹层

```

1 ListItem() {
2 Text('取消')
3 .fontColor(ResManager.EC_MID_59)
4 .fontSize(ResManager.EC_MODULE_TITLE_FS)
5 .fontWeight(500)
6 .width('100%')
7 .textAlign(TextAlign.Center)
8 .padding(15)
9 }
10 .onClick(() => {
11 this.controller.close()
12 })

```

Fence 53

提交代码

# 完成头像上传

4.0的上传巨大的坑点- 无法得到上传之后的地址-上传请求走完，拿不到返回数据

Next版本中进行处理-onheaderReceive 回调中可以拿到返回数据

多线程上传- 致命问题-所有的代码均不能出现的闭包-几乎所有的第三方库 或者自己封装 都或多或少有闭包

```
1 function test () {
2 let a = 1 //被闭包了。gc
3 const add = (b: number) => {
4 return a + b
5 }
6 return add
7 }
```

Fence 54

## basic模块

- 在上传组件定义回调函数

```
1 callback: (url: string) => void = () => {
2 } // 定义回调
```

Fence 55

- 上传组件通过回调将图片地址传出

```
1 // 拍照
2 async takePhoto() {
3 let mContext = getContext(this) as common.Context;
4 try {
5 let pickerProfile: cameraPicker.PickerProfile = {
6 cameraPosition: camera.CameraPosition.CAMERA_POSITION_BACK
7 };
8 let pickerResult: cameraPicker.PickerResult = await
cameraPicker.pick(mContext,
9 [cameraPicker.PickerMediaType.PHOTO,
cameraPicker.PickerMediaType.VIDEO], pickerProfile);
10 if (pickerResult.resultUri) {
11 this.callback(pickerResult.resultUri) // 通过回调传出
12 }
13
14 } catch (error) {
15 let err = error as BusinessError;
16 console.error(`the pick call failed. error code: ${err.code}`);
17 }
18 }
```

Fence 56

```
1 // 选择相册
2 async selectPhoto() {
3 // 选择文件
4 const photoSelectOptions = new picker.PhotoSelectOptions()
5 photoSelectOptions.MIMETYPE = picker.PhotoViewMIMETypes.IMAGE_TYPE
```

```

6 photoSelectOptions.maxSelectNumber = 1
7
8 const photoViewPicker = new picker.PhotoViewPicker()
9 const result = await photoViewPicker.select(photoSelectOptions)
10 // 拿到照片
11 if (result.photoUris?.length) {
12 this.callback(result.photoUris[0]) // 通过回调传出
13 }
14 }

```

Fence 57

- 当选择图片之后，将选择的图片通过组件的回调函数传递到父组件

```

1 // 上传弹层控制
2 uploadDialog: CustomDialogController = new CustomDialogController({
3 // 使用自定义弹层组件
4 builder: EcDialogUpload({
5 callback: (url) => {
6
7 }
8 }),
9 alignment: DialogAlignment.Bottom
10 })

```

Fence 58

图片准备好之后，将获取的图片使用接口上传得到上传成功的地址

- 封装上传 api 函数

### basic模块

:::info

在basic模块下的api目录中新建一个Common.ets文件

:::

```

1 // 使用axios上传 直接调用方法即可 不需要使用upload
2 import { FormData } from '@ohos/axios'
3
4 import { request } from '../utils/Request'
5
6 // 上传接口
7 export function uploadApi(data: FormData): Promise<string> {
8 return request<string, FormData>({
9 url: '/common/upload',
10 method: 'post',
11 headers: {
12 "Content-Type": "multipart/form-data"
13 },
14 data,
15 context: getContext() // 上下文
16 })
17 }

```

Fence 59

- 在api/index.ets中进行统一导出

```
1 | export * from './Common'
```

Fence 60

因为上传文件会考虑到网络及图片大小等因素，所以为了不出现空白，可以在上传间隙弹出一个进度条，等到上传完毕，将进度条进行关闭

- 声明一个自定义弹窗的进度条，使用自己封装的第三方组件库的EcLoading组件

```
1 | // 全屏 loading 弹层
2 | loading: CustomDialogController = new CustomDialogController({
3 | builder: EcLoading({ tip: '上传中...' }),
4 | customStyle: true,
5 | autoCancel: false,
6 | alignment: DialogAlignment.Center
7 | })
8 |
9 | upload(uri: string) {
10 | try {
11 | this.loading.open()
12 |
13 |
14 | } catch (e) {
15 | this.loading.close()
16 | Log.error(e)
17 | }
18 | }
```

Fence 61

- 实现一个上传方法-在上传开始时打开进度条
- 实现上传方法

首选要将图片进行沙箱文件处理，因为上传接口只能接受沙箱文件的内容

```
1 | // 上传方法
2 | async upload(url: string) {
3 | if (url) {
4 | // 1.拷贝到沙箱 2.调用上传接口 得到上传之后的地址
5 | const file = fileIo.openSync(url, fileIo.OpenMode.READ_ONLY)
6 | // 生成一个目标文件的路径
7 | const destPath = getContext().cacheDir + "/" + file.name
8 | // internal://cache:
9 | fileIo.copyFileSync(file.fd, destPath) // 拷贝到沙箱
10 | const data = new FormData()
11 | // 加上file的参数
12 | data.append("file", `internal://cache/${file.name}`)
13 | const res = await uploadApi(data) // 只是传到服务器了 但是并没有更改头像
14 | // 修改头像地址
15 | await updateUserAvatar(res) // 修改用户头像
16 | promptAction.showToast({ message: '修改用户头像成功' })
17 | this.uploadDialog.close() // 关闭弹层
18 | }
19 | }
```

Fence 62

请注意：目前我们只是上传到服务器一张图片，改掉了在当前本地的头像，但是我们个人信息的真实地址并没有发生变化，此时需要进行调用接口更新

- 封装修改用户头像的API(-basic模块)

```
1 /**
2 * 更新用户头像
3 * @param avatar
4 * @returns
5 */
6 export function updateUserPhotoApi(avatar: string): Promise<null> {
7 return request<null, UpdateUserParamsModel>({
8 url: '/employee/user/updateAvatar', method: 'put', params: {
9 avatar
10 }
11 })
12 }
```

Fence 63

- 在上传图片成功之后调用接口进行修改

```
1 // 上传方法
2 async upload(url: string) {
3 if (url) {
4 // 1.拷贝到沙箱 2.调用上传接口 得到上传之后的地址
5 const file = fileIo.openSync(url, fileIo.OpenMode.READ_ONLY)
6 // 生成一个目标文件的路径
7 const destPath = getContext().cacheDir + "/" + file.name
8 // internal://cache:
9 fileIo.copyFileSync(file.fd, destPath) // 拷贝到沙箱
10 const data = new FormData()
11 // 加上file的参数
12 data.append("file", `internal://cache/${file.name}`)
13 const res = await uploadApi(data) // 只是传到服务器了 但是并没有更改头像
14 // 修改头像地址
15 await updateUserAvatar(res) // 修改用户头像
16 promptAction.showToast({ message: '修改用户头像成功' })
17 this.uploadDialog.close() // 关闭弹层
18 }
19 }
```

Fence 64

12:52

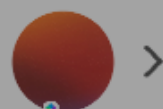
100%

## 个人设置

头像

老哥 >

昵称



上传中...

拍照



提交代码

# 修改用户昵称

- 跳转修改昵称页面，传递当前昵称参数

```
1 Text(this.user.nickname || "-")
2 .fontColor(ResManager.EC_MID_59)
3 .fontSize(ResManager.EC_NORMAL_FS)
4 .onClick(() => {
5 router.pushUrl({
6 url: RouterConfig.USER_NAME_URL,
7 params: {
8 name: this.user.nickname
9 }
10 })
11 })
```

Fence 65

- 昵称页面接收参数并回显

```
1 @State
2 nickname: string = ""
3
4 aboutToAppear(): void {
5 const params = router.getParams() as object
6 if (params && params["name"]) {
7 this.nickname = params["name"]
8 }
9 }
10
11 // Text(this.nickname)
12 TextInput({ text: $$this.nickname, placeholder: '请输入昵称' })
13 .height(50)
14 .borderRadius(ResManager.EC_NORMAL_RD)
15 .backgroundColor('#fff')
16 .placeholderColor(ResManager.EC_MID_99)
```

Fence 66

- 根据昵称值有无对提交按钮控制

```
1 Button('保存', { type: ButtonType.Capsule })
2 .margin({ top: 50 })
3 .width(230)
4 .enabled(!this.nickname.trim())
5 .onClick(() => {
6 this.save()
7 })
```

Fence 67

- 封装修改昵称api 函数

```

1 |
2 | /**
3 | * 更新用户昵称
4 | * @param nickname
5 | * @returns
6 | */
7 | export function updateUserApi(nickname: string): Promise<null> {
8 | return request<null, UpdateUserParamsModel>({
9 | url: '/employee/user/updateNickname', method: 'put', params: {
10 | nickname
11 | }
12 | })
13 | }

```

Fence 68

- 执行修改

```

1 | // 提交
2 | async save() {
3 | await updateUserNickname(this.nickname)
4 | getContext().eventHub.emit("updateUser") // 更新
5 | promptAction.showToast({ message: '修改昵称成功' })
6 | router.back()
7 | }

```

Fence 69

- 回到上一个页面同步修改

因为onPageShow只要页面进入就会执行，直接去除原有的aboutToAppear即可，将相同逻辑在onPageShow中执行即可

```

1 |
2 |
3 | onPageShow(): void {
4 | Log.info('页面可见')
5 | this.getUser()
6 | }

```

Fence 70

此时，在Tabs主页中，我的组件并非Page，不具备事件监听条件，此时可以使用线程通信工具emitter来实现

1:09

100%

我的

该组件非页面



老哥天

45114558@qq.com

设备管理



设备安装



设备绑定



我的设备



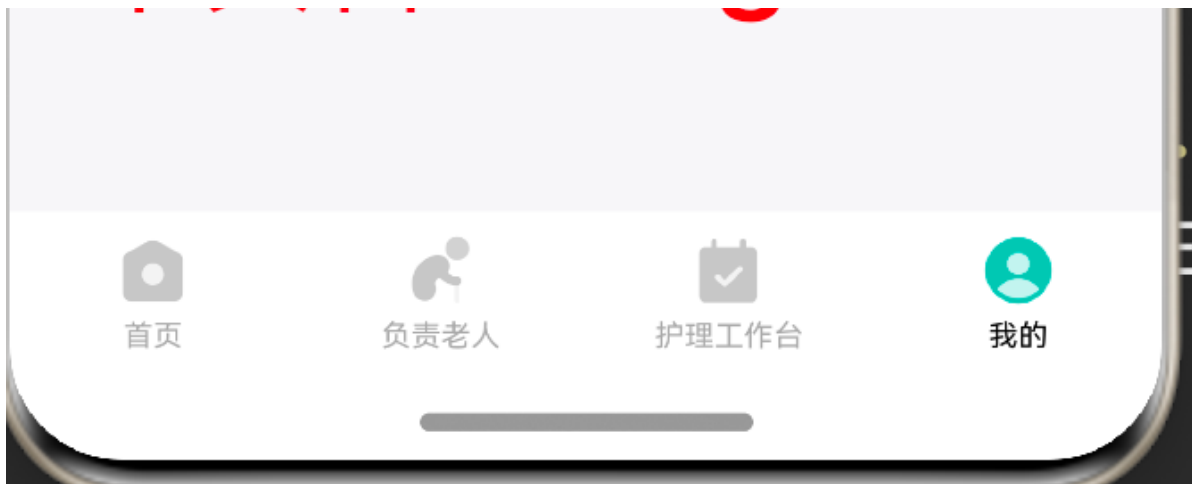
报警设备

我的消息



退出

不具备onPageShow



- 在修改昵称之后发布事件，在我的页面订阅事件
- 发布事件

```
1 // 提交
2 async save() {
3 await updateUserNickName(this.nickname)
4 getContext().eventHub.emit("updateUser") // 更新
5 promptAction.showToast({ message: '修改昵称成功' })
6 router.back()
7 }
```

Fence 71

- 在我的页面订阅事件

```
1 aboutToAppear(): void {
2 this.getUser()
3 getContext()
4 .eventHub.on("updateUser", () => {
5 this.getUser()
6 })
7 }
```

Fence 72

提交代码

# 退出登录

退出登录

提示确认

删除token

跳转到登录页

清除页面栈

```
1 logout() {
2 AlertDialog.show(
3 {
4 title: '提示',
5 subtitle: '',
6 message: '您确认退出登录吗',
7 autoCancel: true,
8 alignment: DialogAlignment.Bottom,
9 gridCount: 4,
10 offset: { dx: 0, dy: -20 },
11 primaryButton: {
12 value: '取消',
13 action: () => {
14
15 }
16 },
17 secondaryButton: {
18 enabled: true,
19 defaultFocus: true,
20 style: DialogButtonStyle.HIGHLIGHT,
21 value: '确定',
22 action: async () => {
23 // 删除token
24 await TokenManager.delUser()
25 router.clear() // 清空页面栈
26 // 跳转到登录栈
27 router.replaceUrl({
28 url: RouterConfig.LOGIN_URL
29 })
30 }
31 }
32 }
33)
34 }
35
```

Fence 73

提交代码

# 设备绑定页面-折叠交互



本次课程我们讲解门禁系统和鸿蒙应用的交互

- 此次课程的门禁是**黑马程序员嵌入式学科 + 鸿蒙学科**共同研发的学习硬件，
- 调试测试需要**华为真机Next系统 + 黑马的自研门禁硬件**，由于测试条件较为苛刻，所以线上同学无法实现真正的代码测试，更重要的是学习 应用 + 硬件设备的编程思想





当点击查看更多时，要展示所有数据  
数据结构如下：

```

1 [
2 {
3 "productId": "test_b6e6b64930ad",
4 "productName": "test_084b38130633",
5 "deviceList": [
6 {
7 "iotId": "test_310f44adaae5",
8 "nodeId": "test_ca719f485de1",
9 "deviceName": "test_9681b00eafa6"
10 }
11]
12 }
13]

```

Fence 74

实际上设备列表是**两层**的结构，所以需要**两层循环**

```

}) {
 ForEach(this.list, (item: UnBindProductItem) => {
 ListItem() {
 this.productItem(item)
 }
 })
}

// 数据超出 6 同时没有点击查看更多
if (item.deviceList?.length! > 6 && !item.isExpand) {...} else {
 ForEach(item.deviceList, (device: DeviceItem) => {
 GridCol() {
 Column() {
 Text(device.deviceName)
 .fontColor(device.isSelected ? '#fff' : ResManager.EC_MID_19)
 .fontSize(ResManager.EC_SMALL_FS)
 }
 .width('100%')
 .padding({
 left: 15

```

:::warning

展开折叠，核心思想就是改变item的isExpand属性，只要能够保证该属性变化，然后驱动响应式更新UI即可

:::

- 实现toggle方法

```

1 @State
2 refreshNum: number = 0
3
4 // 2. 产品选择和折叠
5 // 加载更多和收起
6 toggle(item: UnBindProductItem) {
7 item.isExpand = !item.isExpand
8 this.refreshNum++
9 }

```

Fence 75

- 使用数字作为ForEach中key的一部分，强制UI更新

```

1 ForEach(this.list, (item: UnBindProductItem) => {
2 ListItem() {
3 this.productItem(item)
4 }
5 }, (item: UnBindProductItem) =>
`_${JSON.stringify(item)}_${this.refreshNum}`)

```

Fence 76



...warning

我们要知道鸿蒙只能监听到第一层的数据变化，所以我们巧妙的使用一个随时在变化的key作为ForEach的key，这样只要点击了折叠或者展开，那么key发生了变化，UI就会重新更新，达到我们期望的效果

...

提交代码

# 获取未绑定设备

- 抽提未绑定设备类型

```
1 // 2. 未绑定设备列表
2 export interface DeviceItem {
3 iotId: string
4 nodeId: string
5 deviceName: string
6 isSelected: boolean
7 }
8
9 export interface UnBindProductItem {
10 productId: string
11 productName: string
12 // 是否展开: false 没有点击查看更多 | true 点击了查看更多
13 isExpand: boolean
14 deviceList: DeviceItem[]
15 }
16 export class DeviceItemModel implements DeviceItem {
17 iotId: string = ''
18 nodeId: string = ''
19 deviceName: string = ''
20 isSelected: boolean = false
21
22 constructor(model: DeviceItem) {
23 this.iotId = model.iotId
24 this.nodeId = model.nodeId
25 this.deviceName = model.deviceName
26 this.isSelected = model.isSelected
27 }
28 }
29 export class UnBindProductItemModel implements UnBindProductItem {
30 productId: string = ''
31 productName: string = ''
32 isExpand: boolean = false
33 deviceList: DeviceItem[] = []
34
35 constructor(model: UnBindProductItem) {
36 this.productId = model.productId
37 this.productName = model.productName
38 this.isExpand = model.isExpand
39 this.deviceList = model.deviceList
40 }
41 }
42
```

Fence 77

注意：如果只有interface，可以使用i2c 一键转化

- 封装API

```

1 import { request } from '@hm/basic'
2 import { UnBindProductItemModel } from '../viewModel'
3
4 /**
5 * iot 平台 api
6 * 查询未绑定设备产品列表
7 * @returns
8 */
9 export function getUnbindDeviceListApi(): Promise<UnBindProductItemModel[]>
10 {
11 return request<UnBindProductItemModel[]>({ url: '/employee/iot/deviceTree'
12 })
13 }

```

Fence 78

- 在api/index.ets中导出

```

1 export * from './DeviceAPI'

```

Fence 79

- 在bindDevice页面中请求数据

```

1 // 声明数据
2 @State
3 list: UnBindProductItemModel[] = []

```

Fence 80

- 在EcList组件中的load事件中加载数据

```

1 load = async () => {
2 // 请求数据
3 this.list = await getUnbindDeviceListApi()
4 }

```

Fence 81

- 替换所有的interface类型为class类型

```

List({
 space: 10
}) {
 ForEach(this.list, (item: UnBindProductItemModel) => {
 ListItem() {
 this.productItem(item)
 }
 }, (item: UnBindProductItemModel) => `${JSON.stringify(item)}_${this.
 }
 .width('100%')

```

这里可能又会有同学有这种疑问？

interface和class到底什么时候用？区别是什么？

回答：原则上都没问题，但是interface只能用作类型，class不但可以用作类型，还可以用来构造，当成实际的对象来使用，另外当使用Observed的时候必须使用class，除此之外，没什么区别，但是官方更建议我们使用class，因为class可以进行构造，更方便赋值，至于用interface和class取决于每个公司的开发规范



提交代码

# 选择设备实现

- 选择某个设备有UI效果
- 只有选择了设备才可以进行下一步确定
- 要记录选择的哪个分类的哪个设备-分类和设备都有被选中选项
- 同时只能选择一个设备

:::success

实现方式和之前的展开折叠一样，赋值某个属性即可。只不过要考虑用户有取消选择的需求

:::

```
1 // 选择某个产品下设备
2 /**
3 *
4 * @param device 当前点击产品下的某个设备
5 */
6 selectDevice(item: UnBindProductItemModel, device: DeviceItemModel) {
7 this.list.forEach(prod => {
8 prod.deviceList.forEach(dev => {
9 if (dev.isSelected) {
10 dev.isSelected = false
11 }
12 })
13 })
14 device.isSelected = !device.isSelected
15 this.refreshNum++
16 }
```

Fence 82

- 定义当前的状态记录当前选择设备

```
1 @State
2 currentProduct: UnBindProductItemModel = new UnBindProductItemModel({} as
UnBindProductItem)
3 @State
4 currentSelect: DeviceItemModel = new DeviceItemModel({} as DeviceItem)
```

Fence 83

- 选择时记录

```
1 // 选择某个产品下设备
2 /**
3 *
4 * @param device 当前点击产品下的某个设备
5 */
6 selectDevice(item: UnBindProductItemModel, device: DeviceItemModel) {
7 this.list.forEach(prod => {
8 prod.deviceList.forEach(dev => {
9 if (dev.isSelected) {
10 dev.isSelected = false
11 }
12 })
13 })
14 }
```

```

14 device.isSelected = !device.isSelected
15 this.currentSelect = device // 记录当前选择的设备
16 this.currentProduct = item
17 this.refreshNum++
18 }

```

Fence 84

- 控制确认按钮是否可点击

```

1 Button('确认')
2 .width(312)
3 .backgroundColor(ResManager.EC_MAIN_COLOR)
4 .enabled(!this.currentSelect.iotId) // 判断是否有lotId来决定按钮是否可点击

```

Fence 85



提交代码

# 设备英文映射到中文

由于设备通信的要求，在给设备命名时需要考虑中文的判断代码的合理性，所以后台实际上以英文命名，但是在客户端展示用户需要简单的中文来识别，所以我们准备一个字典库进行匹配



```
1 import { ProductNameList } from '../constants'
2
3 // 根据英文获取中文的方法
4
5 export function getDeviceMap(name: string) {
6 return ProductNameList.find(item => item.en === name) // 返回的是个对象
7 }
```

- 在utils/index.ets中导出

```
1 | export * from './Device'
```

Fence 87

- 将设备中的英文替换成中文

```
productItem(item: UnBindProductItemModel) {
 Column({ space: 18 }) {
 // 1. 产品分类
 Row() {
 Text(getDeviceMap(item.productName)?.cn)
 .fontColor(ResManager.EC_MID_19)
 .fontSize(ResManager.EC_MODULE_TITLE_FS)
 .fontWeight(500)
 // 点击查看更多后显示
 if (item.isExpand) {
 Text() {
 Span('收起') }
 }
 }
 }
}
```

```
1 | @Builder
2 | productItem(item: UnBindProductItemModel) {
3 | Column({ space: 18 }) {
4 | // 1. 产品分类
5 | Row() {
6 | Text(getDeviceMap(item.productName)?.cn || '--')
7 | .fontColor(ResManager.EC_MID_19)
8 | .fontSize(ResManager.EC_MODULE_TITLE_FS)
9 | .fontWeight(500)
10 | // 点击查看更多后显示
11 | if (item.isExpand) {
12 | Text() {
13 | Span('收起')
14 | ImageSpan($r('app.media.ic_arrow_up'))
15 | .width(12.8)
16 | .height(5.6)
17 | .verticalAlign(ImagespanAlignment.CENTER)
18 | .margin({ bottom: 2 })
19 | }
20 | .fontColor(ResManager.EC_MAIN_COLOR)
21 | .fontSize(ResManager.EC_SMALL_FS)
22 | .onClick(() => {
23 | this.toggle(item)
24 | })
25 | }
26 | }
27 | .width('100%')
28 | .justifyContent(FlexAlign.SpaceBetween)
29 |
30 | // 2. 设备列表
31 | GridRow({ columns: 2, gutter: 10 }) {
32 | // 数据超出 6 同时没有点击查看更多
```

```

33 if (item.deviceList?.length! > 6 && !item.isExpand) {
34 ForEach(item.deviceList?.slice(0, 6), (device: DeviceItemModel,
i: number) => {
35 GridCol() {
36 // column() {
37 if (i < 5) {
38 Column() {
39 // 选择设备
40 Text(device.deviceName)
41 .fontColor(device.isSelected ? '#fff' :
ResManager.EC_MID_19)
42 .fontSize(ResManager.EC_SMALL_FS)
43 }
44 .width('100%')
45 .padding({
46 left: 15,
47 right: 15,
48 top: 10,
49 bottom: 10
50 })
51 .borderRadius(ResManager.EC_NORMAL_RD)
52 .backgroundColor(device.isSelected && i < 5 ?
ResManager.EC_MAIN_COLOR : '#fff')
53 .onClick() => {
54 this.selectDevice(device)
55 }
56 } else {
57 Column() {
58 // 数据超出 6条 显示
59 Text() {
60 Span('查看更多')
61 ImageSpan($r('app.media.ic_arrow_down'))
62 .width(12.8)
63 .height(5.6)
64 .verticalAlign(ImageSpanAlignment.CENTER)
65 .margin({ bottom: 2 })
66 }
67 .fontColor(ResManager.EC_MAIN_COLOR)
68 .fontSize(ResManager.EC_SMALL_FS)
69 }
70 .width('100%')
71 .padding({
72 left: 15,
73 right: 15,
74 top: 10,
75 bottom: 10
76 })
77 .borderRadius(ResManager.EC_NORMAL_RD)
78 .backgroundColor(device.isSelected && i < 5 ?
ResManager.EC_MAIN_COLOR : '#fff')
79 .onClick() => {
80 this.toggle(item)
81 }
82 }
83 }

```

```

84 }
85)
86 } else {
87 ForEach(item.deviceList, (device: DeviceItemModel) => {
88 GridCol() {
89 Column() {
90 Text(device.deviceName)
91 .fontColor(device.isSelected ? '#fff' :
ResManager.EC_MID_19)
92 .fontSize(ResManager.EC_SMALL_FS)
93 }
94 .width('100%')
95 .padding({
96 left: 15,
97 right: 15,
98 top: 10,
99 bottom: 10
100 })
101 .borderRadius(ResManager.EC_NORMAL_RD)
102 .backgroundColor(device.isSelected ? ResManager.EC_MAIN_COLOR
: '#fff')
103 .onClick(() => {
104 this.selectDevice(device)
105 })
106 }
107 }
108)
109 }
110
111 }
112
113 }
114 .width('100%')
115 }

```



提交代码

# 检查蓝牙权限

需求分析：使用门禁开门要完成以下几件事

- 检测是否有蓝牙权限-应用的事情
- 检测是否打开蓝牙
- 蓝牙打开-扫描可用设备
- 通过蓝牙连接设备-获取读写数据权限
- 通过蓝牙读写权限给门禁配置wifi账户
- 通过蓝牙连接发送开门指令

:::success

- 检查蓝牙权限

类似于前面同学做过的微信案例，我们检查麦克风权限的流程

为了体现封装的特性，我们单独封装一个蓝牙的检查授权的管理器，因为只在设备中，可以直接在device的hsp中创建，当然也可以放在basic模块中

...

- 配置蓝牙权限

```
1 {
2 "name": "ohos.permission.ACCESS_BLUETOOTH",
3 "reason": "$string:app_name",
4 "usedScene": {
5 "abilities": ["EntryAbility"],
6 "when": "inuse"
7 }
8 }
```

Fence 89

- 新建蓝牙权限管理器

```
1 class BlueManager {
2 // 检查蓝牙授权
3 checkBlue() {
4 }
5 // 检查蓝牙是否打开
6 checkBlueOpen() {
7 }
8 }
9 }
10
11 export const blueManager = new BlueManager()
```

Fence 90

- 在utils/index.ets中导出

```
1 export * from './BlueManager'
```

Fence 91

- 实现检查蓝牙授权方法

```
1 // 检查蓝牙授权
2 async checkBlue() {
3 try {
4 // 1. 获取应用信息
5 const bundleInfo =
6 await
7 bundleManager.getBundleInfoForSelf(bundleManager.BundleFlag.GET_BUNDLE_INFO_
8 WITH_APPLICATION)
9 // 2. 检查蓝牙权限
10 let atManager: abilityAccessCtrl.AtManager =
11 abilityAccessCtrl.createAtManager();
12 let permissionName: Permissions = 'ohos.permission.ACCESS_BLUETOOTH';
13 let data: abilityAccessCtrl.GrantStatus =
14 atManager.checkAccessTokenSync(bundleInfo.appInfo.accessTokenId,
15 permissionName);
16 Log.info('蓝牙权限: ' + data)
17 } catch (err) {
18 let message = (err as BusinessError).message;
19 Log.error(message)
20 }
21 }
```

Fence 92

```
[bundle_manager.cpp(GetBundleInfoComplete:3459)]cache_exit GetBundleInfoComplete
| yk-yl |
| 蓝牙权限: -1 |
390:~WorkerThread:58 WorkerThread enter destruction
```

- 当没有蓝牙权限时，申请蓝牙权限

```
1 // 检查蓝牙权限-应用有没有蓝牙权限
2 async checkBlue() {
3 // 权限的检查
4 const bundleInfo =
5 bundleManager.getBundleInfoForSelfSync(bundleManager.BundleFlag.GET_BUNDLE_I
6 NFO_WITH_APPLICATION)
7 let atManager = abilityAccessCtrl.createAtManager()
8 const status =
9 atManager.checkAccessTokenSync(bundleInfo.appInfo.accessTokenId,
10 "ohos.permission.ACCESS_BLUETOOTH") // 检查是否授权
11 if (status === abilityAccessCtrl.GrantStatus.PERMISSION_DENIED) {
12 // 要去申请蓝牙权限
13 const result = await
14 atManager.requestPermissionsFromUser(getContext(),
15 ["ohos.permission.ACCESS_BLUETOOTH"])
16 // 能否确定授权吗???
17 // AlertDialog.show({ message: result.authResults[0] + "" })
18 if (result.authResults[0] === 0) {
19 // 通过了
20 } else {
21 // 如果不等于0 那就是点击了禁止
22 // 拉起应用系统设置页面
23 }
24 }
```

```

19 let context = getContext(this) as common.UIAbilityContext;
20 context.startAbility({
21 bundleName: 'com.huawei.hmos.settings',
22 abilityName: 'com.huawei.hmos.settings.MainAbility',
23 uri: 'application_info_entry',
24 parameters: {
25 pushParams: 'com.hm.yk.y1' // 应用包名
26 }
27 });
28
29 } // 0 就是通过 -1 是不通过
30
31 }
32 }

```

Fence 93





# 检查蓝牙功能开关

上一步只是检查我们的应用是否有控制蓝牙的权限，这一步我们检查我们的手机系统是否开了蓝牙开关



坑点：模拟器无法检测蓝牙开关

- 封装蓝牙是否打开方法

```
1 // 检查蓝牙是否打开
2 checkBlueOpen() {
3 return access.getState() === access.BluetoothState.STATE_ON
4 }
```

Fence 94

- 授权后检查蓝牙开关是否打开，如果打开正常进行下一步，否则抛出异常

```

1 import abilityAccessCtrl from '@ohos.abilityAccessCtrl'
2 import { bundleManager, common } from '@kit.AbilityKit'
3 import { promptAction } from '@kit.ArkUI'
4 import { access } from '@kit.ConnectivityKit'
5
6 class BlueManager {
7 // 检查蓝牙权限-应用有没有蓝牙权限
8 async checkBlue() {
9 try {
10 // 权限的检查
11 const bundleInfo =
12
13 bundleManager.getBundleInfoForSelfSync(bundleManager.BundleFlag.GET_BUNDLE_
14 INFO_WITH_APPLICATION)
15 let atManager = abilityAccessCtrl.createAtManager()
16 const status =
17 atManager.checkAccessTokensSync(bundleInfo.appInfo.accessTokenId,
18 "ohos.permission.ACCESS_BLUETOOTH") // 检查是否授权
19 if (status === abilityAccessCtrl.GrantStatus.PERMISSION_DENIED) {
20 // 要去申请蓝牙权限
21 const result = await
22 atManager.requestPermissionsFromUser(getContext(),
23 ["ohos.permission.ACCESS_BLUETOOTH"])
24 // 能否确定授权吗???
25 // AlertDialog.show({ message: result.authResults[0] + "" })
26 if (result.authResults[0] === 0) {
27 // 通过了
28 if (!this.checkBlueOpen()) {
29 // 应该打开蓝牙开关 检测开关是关着的
30 return Promise.reject(new Error("请首先打开蓝牙开关"))
31 }
32 return true // 表示检查权限成功
33
34 } else {
35 // 如果不等于0 那就是点击了禁止
36 // 拉起应用系统设置页面
37 let context = getContext(this) as common.UIAbilityContext;
38 context.startAbility({
39 bundleName: 'com.huawei.hmos.settings',
40 abilityName: 'com.huawei.hmos.settings.MainAbility',
41 uri: 'application_info_entry',
42 parameters: {
43 pushParams: 'com.hm.yk.yl' // 应用包名
44 }
45 });
46 return false // 表示授权失败 跳到了设置页面
47 } // 0 就是通过 -1 是不通过
48 }
49
50 // 已经有蓝牙权限
51 if (!this.checkBlueOpen()) {
52 // 检测开关是关着的
53 return Promise.reject(new Error("请首先打开蓝牙开关"))
54 }
55 return true // 表示授权成功

```

```

51 } catch (error) {
52 return false
53 }
54 }
55
56 // 检查系统的蓝牙是否打开 true/打开 false/关闭
57 checkBlueOpen() {
58 return access.getState() === access.BluetoothState.STATE_ON
59 }
60 }
61
62 export const blueManager = new BlueManager()

```

Fence 95

- 确定按钮处理跳转

```

1 async bindDevice() {
2 try {
3 // 这里面检查蓝牙
4 const isOK = await blueManager.checkBlue() // 检查蓝牙是否授权
5 promptAction.showToast({ message: isOK ? '授权成功' : '授权失败' })
6 if (isOK) {
7 // 进行下一步的通信 扫描蓝牙
8 router.pushUrl({
9 url: RouterConfig.DEVICE_NEAR_URL
10 })
11 }
12 } catch (error) {
13 promptAction.showToast({ message: error.message })
14 }
15 }

```

Fence 96

- 绑定确定按钮点击事件

```

1 Button('确认')
2 .width(312)
3 .backgroundColor(ResManager.EC_MAIN_COLOR)
4 .onClick(() => {
5 this.bindDevice() // 绑定设备
6 })
7 .enabled(!this.currentSelect.iotId)

```

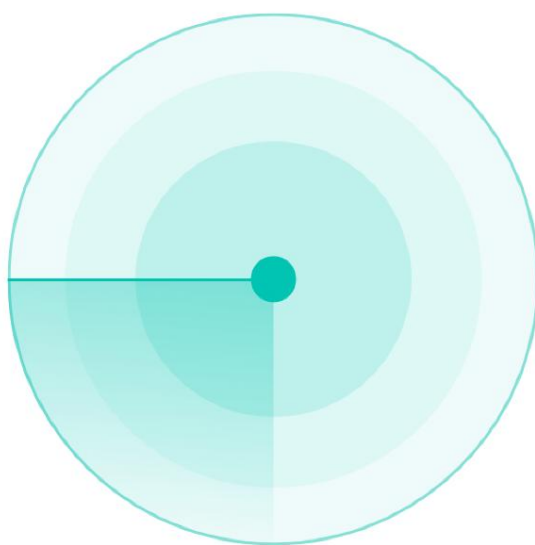
Fence 97

8:33

📶 🔋 100% ⚡



## 附近设备



正在查找附近可连接设备

未找到设备

提交代码

# 记录当前设备的lot信息

我们需要在连接页面使用之前选择设备的lot信息，如lotId-设备id和nodeId-设备标识

但是如果通过页面传递，下次再使用还得一次次的传递，比较麻烦。

可以通过全局状态共享，也可以使用单例控制。全局状态的话因为只属于设备模块，所以我们还是采用单例模式

- 定义存储信息的类型

```
1 export interface LotParams {
2 // 设备 id
3 iotId: string | null
4 // 设备标识
5 nodeId: string | null
6 // 老人 id
7 elderId: string | null
8 // 测量时间：凌晨0-睡前7
9 dateType: number | null
10 }
11
12 export class LotParamsModel implements LotParams {
13 iotId: string | null = null
14 nodeId: string | null = null
15 elderId: string | null = null
16 dateType: number | null = null
17
18 constructor(model: LotParams) {
19 this.iotId = model.iotId
20 this.nodeId = model.nodeId
21 this.elderId = model.elderId
22 this.dateType = model.dateType
23 }
24 }
25
26 // 导出当前单例
27 export const currentLot = new LotParamsModel({} as LotParams)
```

Fence 98

- 在viewmodels/index.ets中导出

```
1 export * from './LotModel'
```

Fence 99

- 在跳转连接蓝牙之前进行赋值

```
1 async bindDevice() {
2 try {
3 // 这里面检查蓝牙
4 const isOK = await blueManager.checkBlue() // 检查蓝牙是否授权
5 promptAction.showToast({ message: isOK ? '授权成功' : '授权失败' })
6 if (isOK) {
7 currentLot.nodeId = this.currentDevice.nodeId // 记录nodeId
8 currentLot.iotId = this.currentDevice.iotId // 记录当前的iotId
```

```
9 // 进行下一步的通信 扫描蓝牙
10 router.pushUrl({
11 url: RouterConfig.DEVICE_NEAR_URL
12 })
13 }
14 } catch (error) {
15 promptAction.showToast({ message: error.message })
16 }
17 }
```

Fence 100

提交代码

# 封装蓝牙管理器

## 封装设备蓝牙操作：

1. 设备扫描
2. 设备连接
3. 设备通信

- 蓝牙开发概述

官方文档：<https://developer.huawei.com/consumer/cn/doc/harmonyos-guides-V5/bluetooth-overview-V5>

## 模块介绍

- a2dp模块（高级音频分发配置文件）：A2DP是Advanced Audio Distribution Profile的缩写，即高级音频分发配置文件。它是一种蓝牙协议，允许无线传输高品质音频流，例如音乐或语音通话，同时支持双向通信，因此可以用于耳机、扬声器、汽车音响等设备。详情请参考[@ohos.bluetooth.a2dp API参考](#)。
  - ble模块（低功耗蓝牙）：BLE是Bluetooth Low Energy的缩写，意为“低功耗蓝牙”。它是一种能够在低功耗情况下进行通信的蓝牙技术，与传统蓝牙相比，BLE的功耗更低，适用于需要长时间运行的低功耗设备，如智能手表、健康监测设备、智能家居等。详情请参考[@ohos.bluetooth.ble API参考](#)。
  - hfp模块（免提模式）：HFP模块是指蓝牙耳机或车载蓝牙设备中的Hands-Free Profile，即“免提模式”。HFP允许用户通过蓝牙连接手机或其他蓝牙设备，实现免提通话和语音控制等功能。详情请参考[@ohos.bluetooth.hfp API参考](#)。
  - hid模块（人机接口设备）：HID是Human Interface Device的缩写，即人机接口设备。在蓝牙中，HID模块是一种允许用户通过蓝牙连接键盘、鼠标、游戏手柄等人机接口设备的模块。用户可以通过HID模块将这些设备连接到蓝牙主机上，实现无线控制和输入。详情请参考[@ohos.bluetooth.hid API参考](#)。
  - pan模块（个人区域网络）：PAN（Personal Area Network）是个人区域网络的缩写，是一种无线通信技术，用于将设备连接到一个小范围的网络中。例如可以将手机、电脑、打印机等设备连接到一个PAN网络中，实现文件的共享和打印。详情请参考[@ohos.bluetooth.pan API参考](#)。
  - gatt模块（通用属性）：GATT是指蓝牙技术中的通用属性（Generic Attribute），它是一种用于在蓝牙低功耗设备之间传输数据的协议。GATT协议定义了一套通用的属性和服务框架，用于描述蓝牙设备之间的通信，同时蓝牙设备可以向其他设备提供服务，也可以从其他设备获取服务。详情请参考[@ohos.bluetooth.ble API参考](#)。
  - spp模块（串口协议）：SPP是Serial Port Profile（串口协议）的缩写，是一种蓝牙协议，用于在蓝牙设备之间建立串行通信连接。通过SPP，蓝牙设备可以像使用串口一样进行数据传输，例如传输文件、文本等。详情请参考[@ohos.bluetooth.socket API参考](#)。
- 首选保证我们在主hap包中开启了蓝牙的控制权限(前面已经设置过)

```

 },
 {
 "name": "ohos.permission.ACCESS_BLUETOOTH",
 "reason": wing_care_hsp,
 "usedScene": {
 "abilities": ["EntryAbility"],
 "when": "inuse"
 }
 }
]
}

```

- 在之前的BlueManager中添加一个扫描设备的方法

```

1 // 扫描设备
2 scanDevice() {
3
4 }

```

Fence 101

```

3 import { BusinessError } from '@kit.BasicServicesKit';
4 import { promptAction } from '@kit.ArkUI';
5 import { access } from '@kit.ConnectivityKit';
6
7 class BlueManager {
8 // 检查蓝牙授权
9 async checkBlue(): Promise<void> {...}
10
11 // 检查蓝牙是否打开
12 checkBlueOpen() {
13 return access.getState() === access.BluetoothState.STATE_ON
14 }
15
16 // 扫描设备
17 scanDevice() {
18
19 }
20 }
21
22 export const blueManager = new BlueManager()

```

我们应用和门禁通信采用的低功耗蓝牙，所以需要使用ble的功能模块

文档地址: <https://developer.huawei.com/consumer/cn/doc/harmonyos-references-V5/js-apis-bluetooth-ble-V5#blestartblescan>



## ble.startBLEScan

startBLEScan(filters: Array<ScanFilter>, options?: ScanOptions): void

发起BLE扫描流程。

需要权限：ohos.permission.ACCESS\_BLUETOOTH

元服务API：从API version 12开始，该接口支持在元服务中使用。

系统能力：SystemCapability.Communication.Bluetooth.Core。

参数：

| 参数名     | 类型                | 必填 | 说明                                  |
|---------|-------------------|----|-------------------------------------|
| filters | Array<ScanFilter> | 是  | 表示扫描结果过滤策略集合，如果不使用过滤的方式，该参数设置为null。 |
| options | ScanOptions       | 否  | 表示扫描的参数配置，可选参数。                     |

### ScanFilter

扫描过滤参数。

元服务API：从API version 12开始，该接口支持在元服务中使用。

系统能力：SystemCapability.Communication.Bluetooth.Core。

| 名称                          | 类型     | 必填 | 说明                                                          |
|-----------------------------|--------|----|-------------------------------------------------------------|
| deviceId                    | string | 否  | 表示过滤的BLE设备地址，例如："XX:XX:XX:XX:XX:XX"。                        |
| name                        | string | 否  | 表示过滤的BLE设备名。                                                |
| serviceUuid                 | string | 否  | 表示过滤包含该UUID服务的设备，例如：00001888-0000-1000-8000-00805f9b34fb。   |
| serviceUuidMask             | string | 否  | 表示过滤包含该UUID服务掩码的设备，例如：FFFFFFFF-FFFF-FFFF-FFFFFFFFFFFFFF。    |
| serviceSolicitationUuid     | string | 否  | 表示过滤包含该UUID服务请求的设备，例如：00001888-0000-1000-8000-00805F9B34FB。 |
| serviceSolicitationUuidMask | string | 否  | 表示过滤包含该UUID服务请求掩码的设备，例如：FFFFFFFF-FFFF-FFFF-FFFFFFFFFFFFFF。  |
|                             |        |    | 表示过滤包含该服务请求掩码的设备。                                           |

### ScanOptions

扫描的配置参数。

元服务API：从API version 12开始，该接口支持在元服务中使用。

系统能力：SystemCapability.Communication.Bluetooth.Core。

| 名称       | 类型       | 可读 | 可写 | 说明                              |
|----------|----------|----|----|---------------------------------|
| interval | number   | 是  | 是  | 表示扫描结果上报延迟时间，默认值为0。             |
| dutyMode | ScanDuty | 是  | 是  | 表示扫描模式，默认值为SCAN_MODE_LOW_POWER。 |

|                        |           |   |   |                                        |
|------------------------|-----------|---|---|----------------------------------------|
| matchMode              | MatchMode | 是 | 是 | 表示硬件的过滤匹配模式，默认值为MATCH_MODE_AGGRESSIVE。 |
| phyType <sup>12+</sup> | PhyType   | 是 | 是 | 表示扫描中使用的PHY类型。                         |

示例:

```
import { AsyncCallback, BusinessError } from '@kit.BasicServicesKit';
function onReceiveEvent(data: Array<ble.ScanResult>) {
 console.info('BLE scan device find result = ' + JSON.stringify(data));
}
try {
 ble.on("BLEDeviceFind", onReceiveEvent);
 let scanFilter: ble.ScanFilter = {
 deviceId: "XX:XX:XX:XX:XX:XX",
 name: "test",
 serviceUuid: "00001888-0000-1000-8000-00805f9b34fb"
 };
 let scanOptions: ble.ScanOptions = {
 interval: 500,
 dutyMode: ble.ScanDuty.SCAN_MODE_LOW_POWER,
 matchMode: ble.MatchMode.MATCH_MODE_AGGRESSIVE
 }
 ble.startBLEScan([scanFilter], scanOptions);
} catch (err) {
 console.error('errCode: ' + (err as BusinessError).code + ', errMessage: ' + (err as BusinessError).message)
}
```

- 实现扫描蓝牙方法

```
1 // 扫描设备
2 scanDevice() {
3 try {
4 ble.on("BLEDeviceFind", (list: ble.ScanResult[]) => {
5 Log.info("扫描到当前蓝牙列表: " + JSON.stringify(list))
6 });
7 let scanOptions: ble.ScanOptions = {
8 interval: 500,
9 dutyMode: ble.ScanDuty.SCAN_MODE_LOW_POWER,
10 matchMode: ble.MatchMode.MATCH_MODE_AGGRESSIVE
11 }
12 ble.startBLEScan(null, scanOptions);
13 } catch (err) {
14 Log.error('errCode: ' + (err as BusinessError).code + ', errMessage: '
15 + (err as BusinessError).message);
16 }
17 }
```

Fence 102

扫描到设备后将扫描后的设备通过回调函数进行回传，并对数据进行重复处理，回传之后，去掉扫描监听

- 代码

```
1 timer: number = -1 // 定义定时器
2
3 // 扫描蓝牙
```

```

4 scanDevice(callback: (res: ble.ScanResult[]) => void) {
5 try {
6 clearTimeout(this.timer) // 清除上一次的 重新开始
7 // 当找到蓝牙时会触发
8 const resList: ble.ScanResult[] = [] // 汇总所有的发现蓝牙
9 ble.on("BLEDeviceFind", (list: ble.ScanResult[]) => {
10 Log.info("扫描到的蓝牙列表: " + JSON.stringify(list))
11 // [1,2,3] [2]
12 resList.push(...list.filter(dv => !resList.some(item => item.deviceId
=== dv.deviceId)))
13 // 告诉页面
14 callback(resList) // 回传汇总的数据
15 });
16 let scanOptions: ble.ScanOptions = {
17 interval: 500,
18 dutyMode: ble.ScanDuty.SCAN_MODE_LOW_POWER,
19 matchMode: ble.MatchMode.MATCH_MODE_AGGRESSIVE
20 }
21 ble.startBLEScan(null, scanOptions);
22 this.timer = setTimeout(() => {
23 ble.stopBLEScan() // 停止扫描
24 ble.off("BLEDeviceFind") // 停止监听该事件
25 }, 5000)
26 } catch (err) {
27 Log.error('errorCode: ' + (err as BusinessError).code + ', errorMessage: '
+ (err as BusinessError).message);
28 }
29 }

```

Fence 103

- 在扫描页面进行处理

```

1 @State
2 list: ble.ScanResult[] = []
3
4 aboutToAppear(): void {
5 this.reset()
6 }
7
8 // 3. 搜索方法
9 search() {
10 blueManager.scanDevice((res) => {
11 if (res.length) {
12 this.hasDevice = true // 发现设备
13 this.list = res
14 }
15 })
16 }

```

Fence 104



```
// 设备列表
```

```
List({ space: 10 }) {
 ForEach(this.list,
 (item: ble.ScanResult, i: number) => {
 ListItem() {
 Row() {
 Column({ space: 5 }) {
 Text(item.deviceName || "未知设备")
 .fontColor(ResManager.EC_MID_19)
 .fontSize(ResManager.EC_NORMAL_FS)
 Text(item.deviceId)
 .fontColor(ResManager.EC_MID_99)
 .fontSize(ResManager.EC_SMALL_FS)
 }
 .alignItems(HorizontalAlign.Start)

 Image($r('app.media.ic_sbbd_ly'))
 .width(24)
 }
 }
 })
}
```



20:30

← 附近设备

未连接设备 〇

SKEY\_6709

C4:D8:16:51:76:09



SKEY\_6709

C4:D8:16:51:76:09



SKEY\_6709

C4:D8:16:51:76:09



SKEY\_6709

C4:D8:16:51:76:09



SKEY\_6709

C4:D8:16:51:76:09





提交代码

# 搜索动画

当没有搜索结果时，显示动画

- 当开始搜索时，开启动画

```
1
2 // 3. 搜索方法
3 // 3. 搜索方法
4 search() {
5 this.run() // 开始动画
6 blueManager.scanDevice((list) => {
7 this.list = list
8 })
9 setTimeout(() => {
10 if (this.list.length) {
11 this.hasDevice = true
12 }
13 clearInterval(this.timer) // 停止动画
14 }, 5000)
15 }
16
17 run() {
18 this.timer = setInterval(() => {
19 this.angle += this.step
20 }, 1000 / 60)
21 }
```

Fence 105

- 当发现设备后，关掉动画

```
1 @State
2 hasDevice: boolean = false
3
```

Fence 106

- 实现动画效果

```
1 Image(ResManager.IC_FJSB_SS)
2 .width(180)
3 .aspectRatio(1)
4 .rotate({
5 angle: this.angle
6 })
7 .animation({
8 duration: 200
9 })
```

Fence 107

提交代码

# 筛选特定黑马门禁设备

---

12:14

6\* 100%

## < 附近设备

### 未连接设备

未知设备

4E:DF:6F:74:70:77



未知设备

B8:91:C2:33:B7:1B



未知设备

0A:5F:71:1C:0A:92



未知设备

AE:67:F3:E1:E7:AD



JL-ITHEIMA-72FD(BLE)

50:03:D2:81:9D:E3



未知设备

26:FC:01:E0:28:F9



未知设备

60:87:37:53:DA:F7



未知设备

48:36:6A:71:C5:86

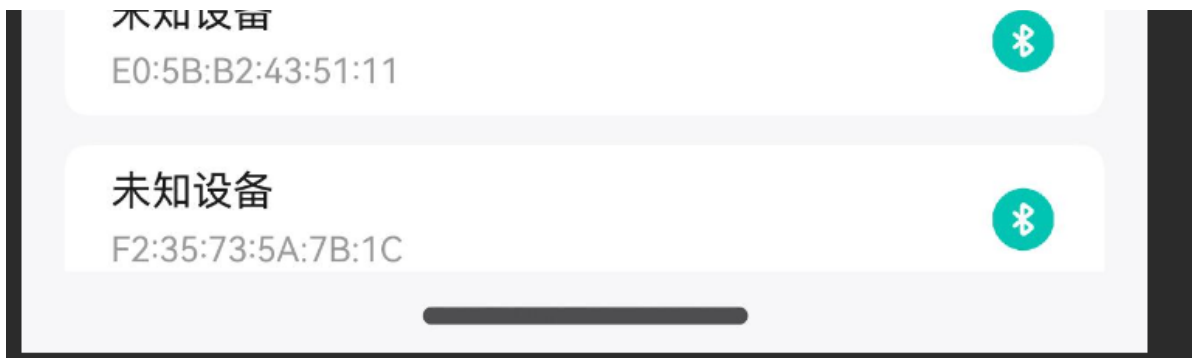


未知设备

76:C1:13:61:44:D8



未知设备



如上图所以，ble扫描出太多无用设备，而黑马门禁有一个特定的前缀JL-ITHEIMA，所以我们直接在数据中筛选一下即可

- 筛选代码

```
1 search() {
2 this.run() // 开始动画
3 blueManager.scanDevice((list) => {
4 this.list = list.filter(item => item.deviceName.startsWith("JL-
ITHEIMA"))
5 })
6 setTimeout(() => {
7 if (this.list.length) {
8 this.hasDevice = true
9 }
10 clearInterval(this.timer) // 停止动画
11 }, 5000)
12 }
```

12:19

6+ 100%



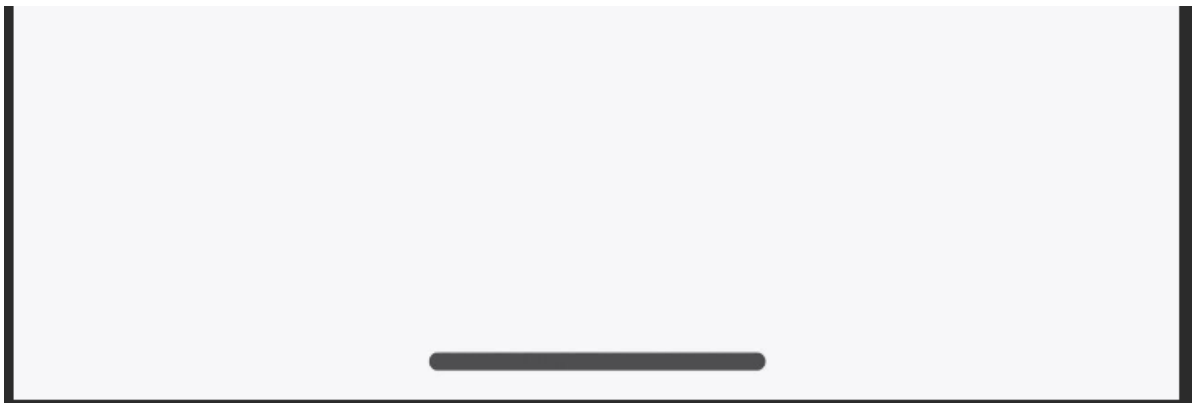
## 附近设备

未连接设备 

JL-ITHEIMA-72FD(BLE)

50:03:D2:81:9D:E3





提交代码

# 连接黑马门禁设备

- 点击特定的黑马门禁设备

ble设备连接文档: <https://developer.huawei.com/consumer/cn/doc/harmonyos-references-V5/js-apis-bluetooth-ble-V5#gattclientdevice>

需求: 连接门禁设备进行开门操作

连接流程:

1. 扫描发现设备
2. 建立连接
3. 获取蓝牙服务 (服务即蓝牙设备提供的功能) => 通信
  1. 通知特征值对象 (获取蓝牙设备发送数据)
  2. 写入特征值对象 (给蓝牙设备发送数据)

- 在blueManager中声明一个方法

```
1 // 连接蓝牙设备
2 connectBlue(deviceId: string) {
3
4 }
```

Fence 109

## 蓝牙服务

在连接蓝牙设备时, 查找**蓝牙服务**是一项重要的步骤。蓝牙服务是蓝牙设备上提供的功能或特性的抽象表示, 它们定义了设备可以提供的各种功能, 比如数据传输、音频播放、传感器数据等。每个**蓝牙服务都包含一个或多个特征** (Characteristics), 这些特征描述了服务所提供的具体功能或属性。

查找蓝牙服务的主要作用包括:

1. 识别设备功能: 通过查找蓝牙服务, 可以了解到连接的蓝牙设备所提供的功能和特性。这有助于应用程序确定如何与设备进行交互, 并选择正确的服务来执行所需的操作。
2. 建立通信通道: 一旦找到了需要的蓝牙服务, 应用程序可以与设备建立通信通道, 以便发送和接收数据。通常, 应用程序需要与设备的特定服务进行通信, 才能执行相应的操作。
3. 配置和控制设备: 通过蓝牙服务, 应用程序可以向设备发送命令或配置参数, 以控制设备的行为或进行设置。例如, 控制蓝牙音箱的音量、切换播放模式等。
4. 获取设备信息: 蓝牙服务通常还包含有关设备的一些信息, 如设备名称、型号、制造商信息等。应用程序可以通过服务获取这些信息, 以便在用户界面中显示或用于其他目的。

总之, **查找蓝牙服务是连接蓝牙设备并与之通信的重要步骤**, 它允许应用程序识别设备功能并与之交互, 从而实现各种有用的功能和应用场景。

# ble.createGattClientDevice

createGattClientDevice(deviceId: string): GattClientDevice

创建一个可使用的GattClientDevice实例。

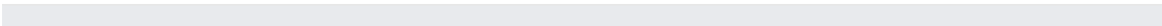
**元服务API：**从API version 12开始，该接口支持在元服务中使用。

**系统能力：**SystemCapability.Communication.Bluetooth.Core。

**参数：**

| 参数名      | 类型     | 必填 | 说明                             |
|----------|--------|----|--------------------------------|
| deviceId | string | 是  | 对端设备地址，例如："XX:XX:XX:XX:XX:XX"。 |

**返回值：**



## 二进制数据操作

- Uint8Array函数

在JavaScript 中，Uint8Array 是一种特殊的类型化数组（TypedArray），用于表示 8 位无符号整数值数组。它提供了一种有效的方式来处理二进制数据，例如图像数据、音频数据等。

重要概念和作用：

1. 数据类型化：Uint8Array 是一种类型化数组，这意味着它包含的数据是一种特定的类型，即 8 位无符号整数。这使得在处理二进制数据时更加高效。
2. 内存分配：与普通数组不同，Uint8Array 使用底层的内存分配机制，可以在创建时分配一块固定大小的内存。这使得它更适合处理大量二进制数据，而无需频繁的内存分配和释放。
3. 索引访问：与普通数组类似，你可以使用索引来访问 Uint8Array 中的元素。例如，myUint8Array[0] 将返回数组中的第一个元素。
4. 二进制数据处理：Uint8Array 常用于处理二进制数据，如图像、音频、视频等。你可以通过 Uint8Array 来读取、修改、传输和存储二进制数据。
5. 与 ArrayBuffer 的关系：Uint8Array 的数据通常存储在 ArrayBuffer 对象中。ArrayBuffer 提供了一种用于在 JavaScript 中表示通用的、固定长度的原始二进制数据缓冲区的方式。Uint8Array 可以视图（view）这个缓冲区，并提供了一种更方便的方式来操作其中的数据。

总之，**Uint8Array 提供了一种高效处理二进制数据的方式**，特别是在需要处理大量数据时。它的使用场景涵盖了诸如图像处理、音频处理、网络通信等各种领域。

- 进制作用和场景

1. **二进制（Binary）：**
  - 是计算机内部最基础的进制方式。
  - 用“0”和“1”两种状态表示电路的开和关。
  - 广泛应用于数字处理、网络通信、加密解密等。
2. **八进制（Octal）：**

- 早期在一些计算机系统中使用，如PDP-8等。
- 由于每个八进制数字代表三个二进制数字，因此在某些情况下可以简化表示。

### 3. 十进制 (Decimal) :

- 日常生活中最常用的进制方式。
- 商业世界中的货币、价格等常用十进制表示。

### 4. 十六进制 (Hexadecimal) :

- 在网络和通信技术中广泛使用，如IP地址。
- 在Web颜色编码、图像处理等领域也有应用。
- 十六进制提供了一种直观且紧凑的表示方式，方便处理底层的二进制数据。

这些不同的进制在计算机科学和技术领域扮演着重要的角色，根据应用场景的不同，选择适合的进制可以提高处理效率和准确性。

说明：重点了解蓝牙设备的连接和通信流程

通信格式：JSON

约定：门禁设备服务serviceUuid以：0000AE30 开头（16）

1. 发现门禁设备服务=》dev.getServices()
2. 根据约定查找服务=》serviceUuid
3. 获取读写特征值对象=》characteristics
  1. 写入特征值以：0000AE10 开头
  2. 通知特征值以：0000AE04 开头

#### • 在connect方法中实现如下流程

- 创建gatt通信实例
- 调用连接方法connect
- 监听连接状态确认连接成功

```

1 currentClient: ble.GattClientDevice | null = null
2
3 // 连接蓝牙
4 connectBlue(deviceId: string, callback: () => void) {
5 if (deviceId) {
6 // 创建gatt的实例对象
7 this.currentClient = ble.createGattClientDevice(deviceId) // 和蓝牙通信
 对象的实例
8 this.currentClient.on("BLEConnectionStateChange", (res) => {
9 if (res.state === constant.ProfileConnectionState.STATE_CONNECTED) {
10 callback() // 连接成功
11 promptAction.showToast({ message: '蓝牙连接成功' })
12 }
13 })
14 this.currentClient.connect() // 连接蓝牙
15 }
16
17 }
```

#### • 控制连接状态

```
1 | @State
2 | isConnected: boolean = false
```

Fence 111

- 在点击右侧蓝牙连接图标时调用

```
1 | Image($r('app.media.ic_sbbsd_ly'))
2 | .width(24)
3 | .aspectRatio(1)
4 | .onClick(() => {
5 | blueManager.connectBlue(item.deviceId, () => {
6 | this.isConnected = true
7 | })
8 | })
```

Fence 112

- 根据状态控制显示状态

```
1 | if (!this.isConnected) {
2 | Row() {
3 | Text('未连接设备')
4 | .fontColor(ResManager.EC_MID_19)
5 | .fontSize(ResManager.EC_MODULE_TITLE_FS)
6 | .fontWeight(500)
7 | LoadingProgress()
8 | .margin({ left: 10 })
9 | .width(20)
10 | .height(20)
11 | .color(ResManager.EC_MID_99)
12 | }
13 | .width('100%')
14 | .margin({ top: 18, bottom: 18 })
15 | }
```

Fence 113

12:47

Bluetooth, Wi-Fi, Cellular, 100%, Battery



附近设备

未连接设备



JL-ITHEIMA-72FD(BLE)

1A:30:D6:9F:20:85





提交代码

# 绑定设备接口调用

调用接口绑定当前的lot门禁设备

- 封装API接口

```
1 /**
2 * 绑定设备
3 * @param iotId 物联网设备id
4 * @returns
5 */
6 export function bindDeviceApi(iotId: string): Promise<null> {
7 return request({ url: `/employee/iot/bindingDevice/${iotId}`, method: 'put'
8 })
9 }
```

Fence 114

- 在连接蓝牙成功后去跳转

```
1 Image($r('app.media.ic_sbbd_ly'))
2 .width(24)
3 .aspectRatio(1)
4 .onClick(() => {
5 try {
6 blueManager.connectBlue(item.deviceId, async () =>
7 {
8 this.isConnected = true // 连接成功
9 // 因为蓝牙连接成功了 所以可以绑定了
10 await bindDeviceApi(currentLot.iotId!) // 物联网
11 // 进入下一步
12 router.pushUrl({
13 url: RouterConfig.DEVICE_BIND_TIP_URL, // 进行
14 params: {
15 tips: 1 // 成功的提示
16 }
17 }) // 连接蓝牙
18 } catch (error) {
19 router.pushUrl({
20 url: RouterConfig.DEVICE_BIND_TIP_URL, // 进行配
21 params: {
22 tips: 2 // 失败的提示
23 }
24 })
25 }
26 })
27 }
```

Fence 115

2:22

6+ 100%



绑定成功

去查看

提交代码

# 获取我的设备

---

## ← 我的设备

### 智能门禁

解绑



smoke alarm 001

解绑



smoke alarm 002

### 智能血糖仪

解绑



Glucose meter 001

解绑



Glucose me... 002

- 封装获取我的设备函数

```
1 /**
2 * 获取我的设备列表
3 * @returns
4 */
5 export function getMyDeviceListApi(): Promise<UnBindProductItemModel[]> {
6 return request<UnBindProductItemModel[]>({ url:
7 `/employee/iot/listDeviceByCurrentUser` })
8 }
9 /**
10 * 解除绑定
11 * **/
12 export function unbindDeviceApi(iotId: string): Promise<null> {
13 return request({ url: `/employee/iot/unBindingDevice/${iotId}`, method:
14 'put' })
15 }
```

Fence 116

- 在我的设备中查询我的设备

```
1 @State
2 devices: UnBindProductItemModel[] = []
3
4 async getMyDevices() {
5 this.devices = await getMyDeviceListApi()
6 }
7
8 aboutToAppear(): void {
9 this.getMyDevices()
10 }
```

Fence 117

- 设备数据渲染

```
1 scroll(this.scroller) {
2 column({ space: 20 }) {
3 ForEach(this.devices, (prod: UnBindProductItemModel) => {
4 column() {
5 // 标题
6 Text(getDeviceMap(prod.productName)?.cn)
7 .fontColor(ResManager.EC_MID_19)
8 .fontSize(ResManager.EC_MODULE_TITLE_FS)
9 .fontWeight(500)
10 .margin({ bottom: 18 })
11 // 内容
12 GridRow({ columns: 2, gutter: { x: 10, y: 10 } }) {
13 ForEach(prod.deviceList, (item: DeviceItemModel, i:
14 number) => {
15 GridCol() {
16 Stack({ alignContent: Alignment.TopEnd }) {
17 column({ space: 10 }) {
```

```

17 Image(getDeviceMap(prod.productName)?.icon)
18 .width(80)
19 .aspectRatio(1)
20 Text(item.deviceName)
21 .fontColor(ResManager.EC_MID_19)
22 .fontSize(ResManager.EC_SMALL_FS)
23 }
24 .width('100%')
25 .padding({ top: 30, bottom: 20 })
26 .borderRadius(8)
27 .backgroundColor(ResManager.EC_MID_WHITE)
28 .onClick(() => {
29
30 })
31
32 Text('解绑')
33 .fontColor(ResManager.EC_MAIN_COLOR)
34 .fontSize(ResManager.EC_SMALL_FS)
35 .margin({ top: 10, right: 10 })
36 .onClick(() => {
37 this.unbind(item)
38 })
39
40 }
41 }
42 .onClick(() => {
43 router.back()
44 })
45 })
46 }
47 }
48 .width('100%')
49 .alignItems(HorizontalAlign.Start)
50 })
51 }
52 .width('100%')
53 }
54 .scrollable(ScrollDirection.Vertical) // 滚动方向纵向
55 .scrollBar(BarState.Off) // 滚动条常驻显示

```

Fence 118

- 设备解绑

```

1 // 解绑
2 unbind(item: DeviceItemModel) {
3 AlertDialog.show({
4 title: '解除确认',
5 message: '确认要解除该设备吗？',
6 primaryButton: {
7 value: '取消',
8 fontColor: ResManager.EC_MID_59,
9 action: () => {
10
11 }
12 },

```

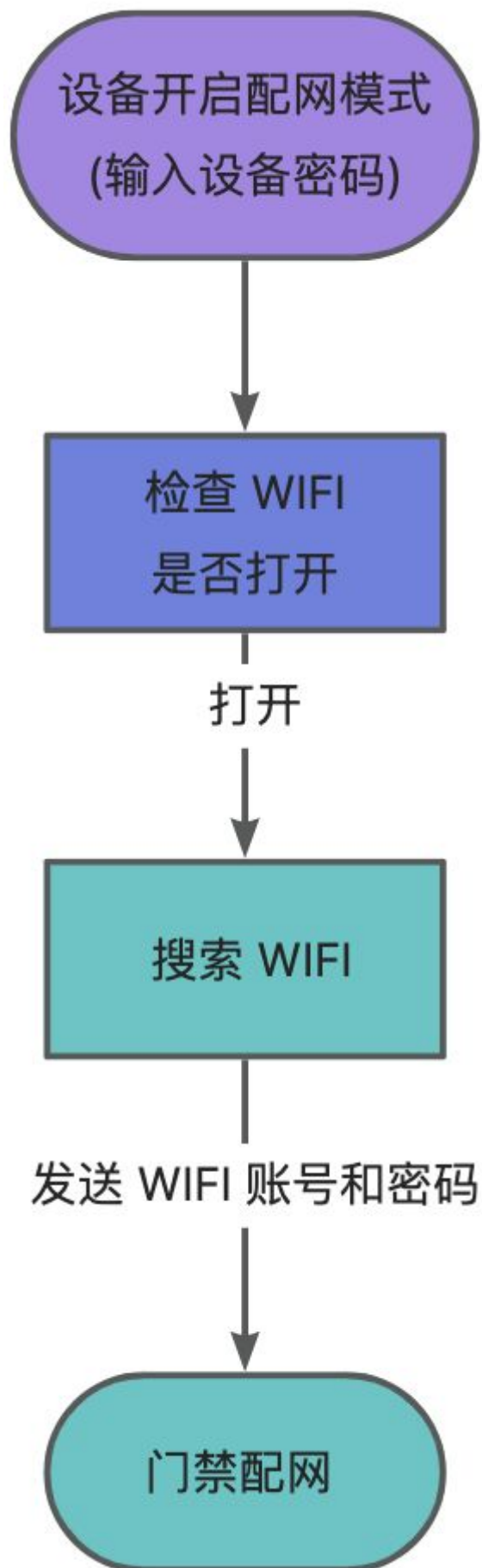
```
13 secondaryButton: {
14 value: '确定',
15 fontColor: ResManager.EC_MAIN_COLOR,
16 action: async () => {
17 await unbindDeviceApi(item.iotId)
18 this.getMyDevices()
19 promptAction.showToast({ message: '解绑成功' })
20 }
21 },
22 alignment: DialogAlignment.Center
23 })
24 }
```

Fence 119

提交代码

# 设备配网-检查wifi状态

需求





请开启以下权限



WIFI

请确认手机已开启WIFI功能

取消

去设置



指示灯闪烁

下一步

未看到指示灯闪烁

- 配置wifi网络权限

```
1 {
2 "name": "ohos.permission.GET_WIFI_INFO"
3 },
```

Fence 120

- 封装一个wifiManager

```
1 class WfManager {
2 }
3
4 export const wfManager = new WfManager()
```

Fence 121

- 在utils/index.ets中导出

```
1 export * from './WfManager'
```

Fence 122

- 实现检查Wifi开关的方法

```
1 import WfManager from '@ohos.WfManager';
2
3 class WfManager {
4 checkWifi() {
5 return WfManager.isWifiActive()
6 }
7 }
8
9 export const WfManager = new WfManager()
```

Fence 123

- 去配网跳转到配网页面

```
1 EcDeviceTip({
2 type: 'success',
3 message: '绑定成功',
4 btnText: '去配网',
5 closeUrl: '',
6 url: RouterConfig.DEVICE_DOOR_NET_URL
7 })
```

Fence 124

- 配网页面检查勾选，确定时检查wifi

```
1 @State
2 isOk: boolean = false
```

Fence 125

- 双向绑定指示灯闪灯

```

1 Checkbox()
2 .select($$this.isOk)
3 .selectedColor(ResManager.EC_MAIN_COLOR)
4 Text('指示灯闪烁')
5 .fontColor(ResManager.EC_MID_19)
6 .fontSize(18)
7 .fontWeight(500)

```

Fence 126

- 实现下一步逻辑

```

1 // 下一步逻辑
2 next() {
3 if (wfManager.checkwifi()) {
4 router.pushUrl({
5 url: RouterConfig.DEVICE_DOOR_NET_CONNECT_URL
6 })
7 } else {
8 this.wifiDialog.open()
9 }
10 }

```

Fence 127

```

Button('下一步')
 .width(230)
 .margin({ bottom: 25 })
 .enabled(this.isOk)
 .backgroundColor(ResManager.EC_MAIN_COLOR)
 .onClick(() => {
 this.next()
 })

```

提交代码

# 搜索wifi渲染列表



- 点击wifi跳转到wifi扫描页

```
1 ListItem() {
2 Row() {
3 Text('YJY-5G')
4 .fontColor(ResManager.EC_MID_19)
5 .fontSize(ResManager.EC_MODULE_TITLE_FS)
6 .fontWeight(500)
7 Row() {
8 Image($r('app.media.ic_arrow_black'))
9 .width(7)
10 .height(16)
11 }
12 }
13 .width('100%')
14 .padding({ top: 14, bottom: 14 })
15 .justifyContent(FlexAlign.SpaceBetween)
16 .onClick(() => {
17 router.pushUrl({
18 url: RouterConfig.DEVICE_DOOR_SCAN_WIFI_URL
19 })
20 })
21 }
```

```

20 })
21 }

```

Fence 128

- 在扫描页实现扫描wifi的方法

```

1 @State
2 list: wifiManager.wifiScanInfo[] = []
3
4 aboutToAppear(): void {
5 this.scanwifi()
6 }
7
8 scanwifi() {
9 this.list = wifiManager.getScanInfoList()
10 }

```

Fence 129

- 渲染wifi列表

```

1 // 设备列表
2 List({ space: 10 }) {
3 ForEach(
4 this.list,
5 (item: wifiManager.WifiScanInfo, i: number) => {
6 ListItem() {
7 Row() {
8 Column({ space: 5 }) {
9 Text(item.ssid)
10 .fontColor(ResManager.EC_MID_19)
11 .fontSize(ResManager.EC_NORMAL_FS)
12 Text(item.bssid)
13 .fontColor(ResManager.EC_MID_99)
14 .fontSize(ResManager.EC_SMALL_FS)
15 }
16 .alignItems(HorizontalAlign.Start)
17
18 Image(ResManager.IC_MJPW_WIFI)
19 .width(24)
20 .aspectRatio(1)
21 }
22 .width('100%')
23 .justifyContent(FlexAlign.SpaceBetween)
24 .padding({
25 left: 16,
26 right: 16,
27 top: 8,
28 bottom: 8
29 })
30 .borderRadius(ResManager.EC_NORMAL_RD)
31 .backgroundColor('#fff')
32 }
33 })
34 }
35 .layoutweight(1)

```



4:01

6+ 100%

## < 查找WIFI

### WIFI列表

ChinaNet-SmQ9

50:1e:b4:fe:9d:a8



9-32-602

a2:dc:d5:af:4c:a7



水冰月

24:76:14:44:bb:81



Shui

7e:35:52:e5:b5:98



Shui\_Wi-Fi5

b8:87:60:a7:56:00



CMCC-UhV4-5G-FAST

c2:48:d2:7c:b9:3f



ChinaNet-PYja

cc:1e:0f:d6:23:52



TP-LINK\_8E93

d2:27:7d:0a:0b:3d



CMCC-UhV4

e0:99:6b:23:91:d9



TP-LINK\_9ADF

30:97:0a:2a:50:9d



ChinaNet-s9cf

5e:b9:68:5f:ef:34



提交代码

# 选择wifi，绑定表单

- 点击某个wifi，返回wifi的ssid数据

```
1 ListItem()
2
3 .onClick() => {
4 router.back({
5 url: RouterConfig.DEVICE_DOOR_NET_CONNECT_URL,
6 params: {
7 ssid: item.ssid
8 }
9 })
10 })
```

Fence 131

- 在表单页面双向绑定表单

```
1
2 @State
3 wifiAccount: string = ''
4 @State
5 wifiPassword: string = ''
```

Fence 132

- 绑定表单

```

List() {
 ListItem() {
 Row() {
 Text(this.wifiAccount || '--')
 .fontColor(ResManager.EC_MID_19)
 .fontSize(ResManager.EC_MODULE_TITLE_FS)
 .fontWeight(500)
 }
 Row() {
 Image($r('app.media.ic_arrow_black'))
 .width(7)
 .height(16)
 }
 }
}
.width('100%')

```

```

Row() {
 TextInput({ placeholder: '请输入密码', text: $$this.wifiPassword })
 .placeholderColor(ResManager.EC_MID_99)
 .placeholderFont({ size: ResManager.EC_NORMAL_FS })
 .backgroundColor('#fff')
 .width(150)
 .textAlign(TextAlign.End)
 .padding({ right: 20 })
}

```

- 在onPageShow中获取得到的ssid进行赋值

```

1 onPageShow(): void {
2 const params = router.getParams() as object
3 if (params && params["ssid"]) {
4 this.wifiAccount = params["ssid"]
5 }
6 }

```

4:12

📶 100%



## 为设备连接Wi-Fi

- 1、请确保接入区络可正常使用
- 2、请勿连接公共Wi-Fi、办公Wi-Fi等需要短信验证码或页面登录的Wi-Fi
- 3、请勿连提名称前有“5G”的Wi-Fi

Shui



Wi-Fi-密码

请输入密码

下一步



提交代码

# 发送wifi账号和密码给门禁

发送 WiFi 账号给设备：

1. 读写特征值：writeCharacteristicValue =》发送数据
2. 通知特征值：BLECharacteristicChange事件 =》接收数据

- 定义和门禁通信数据格式

```
1 // 1. 蓝牙开门 2. 配置设备 wifi 连网
2 export interface BlueData {
3 status?: 200 | 400 // 200 成功 400 失败
4 msg?: string // 消息提示
5 command?: 'open' | 'wifi'
6 data?: string[] // 例如wifi=》 [ssid,pwd]
7 }
```

Fence 134

注意：该数据格式由硬件工程师和前端工程师共同商定

- 首先将之前的蓝牙连接改造成持久化状态

经过测试，蓝牙的实例对象创建之后，如果为局部变量，局部变量被回收之后，那么蓝牙也会断开

```
1 currentClient?: ble.GattClientDevice
2
3 // 连接蓝牙设备
4 connectBlue(deviceId: string, callback: () => void) {
5 if (deviceId) {
6 this.currentClient?.disconnect()
7 this.currentClient = ble.createGattClientDevice(deviceId) // 创建实例
8 this.currentClient.on("BLEConnectionStateChange", (res) => {
9 if (res.state === constant.ProfileConnectionState.STATE_CONNECTED) {
10 callback()
11 promptAction.showToast({ message: '蓝牙连接成功' })
12 }
13 })
14 this.currentClient.connect()
15 }
16 }
```

Fence 135

- 点击下一步开始连接

```
1 Button('下一步')
2 .margin({ top: 50 })
3 .width(230)
4 .backgroundColor(ResManager.EC_MAIN_COLOR)
5 .enabled(!(this.wifiAccount && this.wifiPassword))
6 .onClick(() => {
7 this.start()
8 })
```

Fence 136

- 组装要发送的数据格式

```
1 start() {
2 this.linking = true
3 this.run()
4 try {
5 // 数据
6 const data: BlueData = {
7 command: 'wifi',
8 data: [this.wifiAccount, this.wifiPassword]
9 }
10
11 } catch (e) {
12 Log.error(e)
13 }
14 }
```

Fence 137

说明：重点了解蓝牙设备的连接和通信流程

通信格式：JSON

约定：门禁设备服务serviceUuid以：0000AE30 开头（16）

1. 发现门禁设备服务=》dev.getServices()
2. 根据约定查找服务=》serviceUuid
3. 获取读写特征值对象=》characteristics
  1. 写入特征值以：0000AE10 开头
  2. 通知特征值以：0000AE04 开头

- 封装发送数据方法

```
1 // 给蓝牙发送数据
2 async sendData(data: BlueData) {
3 try {
4 // 发送数据
5 if (this.currentClient) {
6 const list = await this.currentClient?.getServices()
7 const doorService = list.find(item =>
8 item.serviceUuid.startsWith("0000AE30")) // 找到门禁服务
9 if (doorService) {
10 // 再找门禁服务的读写特征值的对象
11 const writeObj =
12 doorService.characteristics.find(item =>
13 item.characteristicUuid.startsWith("0000AE10")) // 写入特征值
14 if (writeObj) {
15 const encoder = new util.TextEncoder() // 生成一个编码对象
16 const unit8Array = encoder.encodeInto(JSON.stringify(data))
17 // 有了写入特征值 就可以通过蓝牙进行通信了
18 await this.currentClient.writeCharacteristicValue({
19 serviceUuid: writeObj.serviceUuid,
20 characteristicUuid: writeObj.characteristicUuid,
21 characteristicValue: unit8Array.buffer, // 将数据转化成二进制的
22 arrayBuffer
23 descriptors: [] // 要写入的数据,
24 }, ble.GattWriteType.WRITE)
```

```

22 }
23 }
24 }
25 } catch (error) {
26 promptAction.showToast({ message: '发送数据失败, ' + error.message })
27 }
28
29 }

```

Fence 138

- 在连接方法中监听wifi连接成功的命令

```

1 // 连接蓝牙设备
2 connectBlue(deviceId: string, callback: () => void) {
3 if (deviceId) {
4 this.currentClient?.disconnect()
5 this.currentClient = ble.createGattClientDevice(deviceId) // 创建实例
6 this.currentClient.on("BLEConnectionStateChange", async (res) => {
7 if (res.state === constant.ProfileConnectionState.STATE_CONNECTED) {
8 callback()
9 promptAction.showToast({ message: '蓝牙连接成功' })
10 // 监听wifi成功连接
11 this.currentClient?.on("BLECharacteristicChange", (res) => {
12 // 接收蓝牙设备的数据响应
13 const td = util.TextDecoder.create()
14 const buff = new Uint8Array(res.characteristicValue)
15 const result = JSON.parse(td.decodeWithStream(buff)) as BlueData
16 if (result.command === 'wifi') {
17 // 配网的响应处理
18 emitter.emit("wifiConnect", {
19 data: {
20 status: result.status
21 }
22 })
23 }
24 })
25 const serviceList = await this.currentClient?.getServices() // 获取所有服务
26 const ser = serviceList?.find(item =>
27 item.serviceUuid.startsWith('0000AE30'))
28 const messageObj = ser?.characteristics.find(item =>
29 item.characteristicUuid.startsWith('0000AE04'))!
30 await
31 this.currentClient?.setCharacteristicChangeNotification(messageObj, true)
32 }
33 })
34 }
35 this.currentClient.connect()
36 }

```

Fence 139

- 实现联网

```

2 // 开始连接网络
3 async start() {
4 this.linking = true
5 this.run()
6 try {
7 // 数据
8 const data: BlueData = {
9 command: 'wifi',
10 data: [this.wifiAccount, this.wifiPassword]
11 }
12 const isOk = await blueManager.sendData(data) // 发送数据
13 promptAction.showToast({ message: isOk ? "发送账号成功" : "发送账号失败"
14 })
15 } catch (e) {
16 Log.error(e)
17 }
18 aboutToAppear(): void {
19 emitter.on("wifiConnect", (res) => {
20 router.pushUrl({
21 url: RouterConfig.DEVICE_DOOR_CONNECT_TIP_URL,
22 params: {
23 tips: res.data?.status === 200 ? 1 : 2
24 }
25 })
26 })
27 }
28 aboutToDisappear(): void {
29 clearInterval(this.timer)
30 }

```

Fence 140

提交代码

# 设备控制

• 门禁管理

5G 5G 5G

20:30

← 门禁管理

smoke alarm 002

在线



蓝牙开门 >

远程开门 >

临时密码 >

开门记录 >

- 点击我的设备进入

```
1 .onClick(() => {
2 currentLot.iotId = item.iotId
3 router.pushUrl({
4 url: RouterConfig.DEVICE_DOOR_CONTROL_URL,
5 params: {
6 iotId: item.iotId
7 }
8 })
9 })
```

Fence 141

- 封装门禁信息类型

```
1 // 设备状态查询
2 export interface DeviceStatusData {
3 /** 设备名称 */
4 name: string;
5
6 /** 设备状态, 设备状态, ONLINE: 设备在线, OFFLINE: 设备离线, ABNORMAL: 设备异常,
7 INACTIVE: 设备未激活, FROZEN: 设备冻结 */
8 status: string;
9 }
10 export class DeviceStatusDataModel implements DeviceStatusData {
11 name: string = ''
12 status: string = ''
13
14 constructor(model: DeviceStatusData) {
15 this.name = model.name
16 this.status = model.status
17 }
18 }
```

Fence 142

- 查询门禁信息-封装 api 函数

```
1 /**
2 * 获取设备状态
3 * @param iotId 物联网设备id
4 * @returns
5 */
6 export function getDeviceStatusApi(iotId: string):
7 Promise<DeviceStatusDataModel> {
8 return request<DeviceStatusDataModel>({ url:
9 `/employee/iot/getDeviceStatus/${iotId}` })
10 }
```

Fence 143

- 查询数据

```

1 @State
2 deviceStatus: DeviceStatusDataModel = new DeviceStatusDataModel({} as
DeviceStatusData)
3
4 aboutToAppear(): void {
5 this.getDeviceStatus()
6 }
7
8 async getDeviceStatus() {
9 const params = router.getParams() as object
10 if (params && params["iotId"]) {
11 this.deviceStatus = await getDeviceStatusApi(params["iotId"])
12 }
13 }

```

Fence 144

- 渲染数据

```

1 Text(this.deviceStatus?.name)
2 .fontColor(ResManager.EC_MID_19)
3 .fontSize(ResManager.EC_MODULE_TITLE_FS)
4 .margin({ top: 20 })
5 Text(this.deviceStatus.status === "ONLINE" ? "在线" : "离线")
6 .fontColor('#fff')
7 .fontSize(ResManager.EC_SMALL_FS)
8 .padding({
9 left: 14,
10 right: 14,
11 top: 2.5,
12 bottom: 2.5
13 })

```

Fence 145

- 替换跳转控制页面

```

1 list: ControlItem[] = [
2 { name: '蓝牙开门', url: RouterConfig.DEVICE_DOOR_BLUE_OPEN_URL },
3 { name: '远程开门', url: RouterConfig.DEVICE_DOOR_REMOTE_OPEN_URL },
4 { name: '临时密码', url: RouterConfig.DEVICE_DOOR_PASSWORD_URL },
5 { name: '开门记录', url: RouterConfig.DEVICE_DOOR_OPEN_RECORD_URL },
6]

```

Fence 146

- 自动连接蓝牙

```

1 // 是否连接到设备
2 @State
3 isLink: boolean = false
4 // 全屏 loading 弹层
5 loading: CustomDialogController = new CustomDialogController({
6 builder: EcLoading({ tip: '连接设备中...' }),
7 customStyle: true,
8 autoCancel: false,
9 alignment: DialogAlignment.Center
10 })

```

Fence 147

- 连接门禁

需求：从我的设备页面，点击门禁，进入门禁管理页面，进行设备鉴权和连接，为蓝牙开门准备

1. 请求蓝牙用户权限
2. 搜索门禁设备
3. 连接门禁设备，记录连接状态

```

1 aboutToAppear(): void {
2 this.getDeviceStatus()
3 this.connect()
4 }
5
6 // 连接蓝牙
7 async connect() {
8 this.loading.open()
9 try {
10 const isConnect = await blueManager.checkBlue() // 检查蓝牙
11 if (isConnect) {
12 blueManager.scanDevice((list) => {
13 if (!this.isLink) {
14 const blue = list.find(item => item.deviceName.startsWith("JL-ITHEIMA"))
15 if (blue) {
16 blueManager.connectBlue(blue.deviceId, () => {
17 this.isLink = true
18 this.loading.close()
19 })
20 }
21 }
22 })
23 }
24 } catch (error) {
25 this.loading.close()
26 promptAction.showToast({ message: '没找到设备' })
27 }
28 }

```

Fence 148

上课时的问题，由于连接过蓝牙之后，不能再次扫描到，必须要断开连接之后才可以连接，所以我们需要在返回页面上一个页面时先断掉蓝牙

- 声明一个断开蓝牙的方法

```
1 // 销毁蓝牙连接
2 destroy() {
3 if (this.currentClient) {
4 this.currentClient.disconnect()
5 this.currentClient.close()
6 this.currentClient = null
7 }
8 }
```

Fence 149

- 在控制页面返回时进行断开

```
1 onBackPress(): boolean | void {
2 blueManager.destroy()
3 }
```

Fence 150

提交代码

# 蓝牙开门



20:30

← 蓝牙开门

## 摇一摇开门



- 动画

分析：控制图片的左右晃动，播放声音

- 配置权限

传感器

```

1 "requestPermissions": [
2 // ...
3 {
4 "name": "ohos.permission.ACCELEROMETER"
5 }
6]

```

Fence 151

## 1. 开启动画

```

1 // 摇一摇动画
2 run() {
3 this.timer = setInterval(() => {
4 if (this.angle !== 30) {
5 this.angle = 30
6 } else {
7 this.angle = 0
8 }
9 // 播放音频
10 AVPlayerClass.playAudio("audio_shake.mp3")
11 }, 200)
12 }

```

Fence 152

### • 关闭动画

```

1 stop() {
2 setTimeout(() => {
3 // 关闭动画
4 clearInterval(this.timer)
5 }, 1000)
6 }

```

Fence 153

### • 摇一摇检测

原理：当我们在晃动手机时，手机本身会产生一个加速度。我们可以**获取手机的两次加速度进行比较**，中间会有一个差值，当差值大于某个幅度时就可以认定用户进行了摇一摇这个操作。

**通过传感器模块判定是否是摇一摇动作，执行业务**

#### sensor.on

on(type: SensorId.ACCELEROMETER, callback: Callback, options?: Options): void

订阅加速度传感器数据。

**需要权限：** ohos.permission.ACCELEROMETER

**系统能力：** SystemCapability.Sensors.Sensor

**参数：**

| 参数名      | 类型                                                                                                                                                                                                         | 必填 | 说明                                        |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|-------------------------------------------|
| type     | <a href="#">SensorId.ACCELEROMETER</a>                                                                                                                                                                     | 是  | 传感器类型，该值固定为 SensorId.ACCELEROMETER。       |
| callback | <a href="#">Callback[AccelerometerResponse](http://developer.huawei.com/consumer/cn/doc/harmonyos-references-V5/js-apis-sensor-0000001861887761-V5#ZH-CN_TOPIC_0000001881258313_accelerometerresponse)</a> | 是  | 回调函数，异步上报的传感器数据固定为 AccelerometerResponse。 |
| options  | <a href="#">Options</a>                                                                                                                                                                                    | 否  | 可选参数列表，用于设置传感器上报频率，默认值为 200000000ns。      |

:::tips

- 1. **纳秒**（nanosecond，缩写为ns）是时间单位，表示的是一秒钟的十亿分之一。换算关系如下：  
1 秒 = 1,000,000,000 纳秒
- 2. **m/s<sup>2</sup>**加速度的单位，表示速度在单位时间（秒）内的变化量。例如，如果一个物体原本以5 m/s的速度运动，然后在1秒内加速到10 m/s，那么它的加速度就是5 m/s<sup>2</sup>（因为速度增加了5 m/s）
- ...
- 错误码：**  
以下错误码的详细介绍请参见 [ohos.sensor\(传感器\)错误码](#)。

| 错误码ID    | 错误信息               |
|----------|--------------------|
| 14500101 | Service exception. |

**示例：**

```
1 import sensor from "@ohos.sensor";
2 import BusinessException from "@ohos.base";
3
4 try {
5 sensor.on(sensor.SensorId.ACCELEROMETER, (data:
6 sensor.AccelerometerResponse) => {
7 console.info('Succeeded in invoking on. X-coordinate component: ' +
8 data.x);
9 console.info('Succeeded in invoking on. Y-coordinate component: ' +
10 data.y);
11 });
12 }
```

```

8 console.info('Succeeded in invoking on. Z-coordinate component: ' +
data.z);
9 }, { interval: 100000000 });
10 setTimeout(() => {
11 sensor.off(sensor.SensorId.ACCELEROMETER);
12 }, 500);
13 } catch (error) {
14 let e: BusinessError.BusinessError = error as BusinessError.BusinessError;
15 console.error(`Failed to invoke on. Code: ${e.code}, message:
${e.message}`);
16 }

```

Fence 154

- 配置权限

```

1 "requestPermissions": [
2 // ...
3 {
4 "name": "ohos.permission.ACCELEROMETER"
5 }
6]

```

Fence 155

- 实现检测

### 1. 监听动力加速度传感器

```

1 // 监听传感器
2 onSensor() {
3 sensor.on(sensor.SensorId.ACCELEROMETER, (data) => {
4 let range = Math.abs(data.x - this.lastX) +
5 Math.abs(data.y - this.lastY) +
6 Math.abs(data.z - this.lastZ)
7
8 // range是三个方向移动的范围 超过了 50就认为摇一摇了
9 if (range > 50) {
10 // 此时开始摇一摇 当它停止的时候 才进行逻辑
11 this.isShake = true
12 }
13 if (this.isShake && range < 10) {
14 // 此时认为 开始了摇一摇 并且停止了摇一摇
15 // 可以动画 可以声音 可以开门了
16 this.isShake = false // 再次等待下一摇一摇
17 this.openDoor()
18 }
19
20 // x y z
21 this.lastX = data.x
22 this.lastY = data.y
23 this.lastZ = data.z
24 })
25 }
26

```

Fence 156

2. 定义摇一摇检测状态
3. 定义摇一摇检测方法并调用

1. 获取手机两次加速度差值range

1.  $\text{Math.abs}(\text{data.x} - \text{this.lastX}) + \text{Math.abs}(\text{data.y} - \text{this.lastY}) + \text{Math.abs}(\text{data.z} - \text{this.lastZ})$

2. range 大于startRange阈值, 开始摇动手机了

1. isShake: true

3. range 小于endRange同时上一次已经开始摇动手机了=> 摇一摇完成, 执行业务

1. isShake: false

4. 记录最后一次摇一摇加速度

- 摇一摇逻辑

```
1 // 监听传感器
2 onSensor() {
3 sensor.on(sensor.SensorId.ACCELEROMETER, (data) => {
4 let range = Math.abs(data.x - this.lastX) +
5 Math.abs(data.y - this.lastY) +
6 Math.abs(data.z - this.lastZ)
7
8 // range是三个方向移动的范围 超过了 50就认为摇一摇了
9 if (range > 50) {
10 // 此时开始摇一摇 当它停止的时候 才进行逻辑
11 this.isShake = true
12 }
13 if (this.isShake && range < 10) {
14 // 此时认为 开始了摇一摇 并且停止了摇一摇
15 // 可以动画 可以声音 可以开门了
16 this.isShake = false // 再次等待下一摇一摇
17 this.openDoor()
18 }
19
20 // x y z
21 this.lastX = data.x
22 this.lastY = data.y
23 this.lastZ = data.z
24 })
25 }
26
```

Fence 157

说明: 需要调整检测事件参数

- 摇一摇开门

```
1 async openDoor() {
2 this.run()
3 await blueManager.sendData({
4 command: "open"
5 })
6 this.stop()
7 promptAction.showToast({ message: '开门成功' })
8 }
```

Fence 158

提交代码